

UNIVERSITY OF SOUTHAMPTON
Faculty of Engineering and Physical Sciences
School of Electronics and Computer Science

A project report submitted for the award of
BSc Computer Science

Supervisor: Dr. Christine Evers
Examiner: Prof. Age Chaplin

**Interpretability of Linear
Time-Variant State Space Sequence
Models**

by **Lok Hei Lam**

April 28, 2026

Statement of Originality

- I have read and understood the ECS Academic Integrity information and the University's Academic Integrity Guidance for Students.
- I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have not used any resources produced by anyone else.

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible

repositories e.g. Github, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

I did all the work myself, or with my allocated group, and have not helped anyone else.

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report, or submitted as a separate file) so that your results could be reproduced.

The material in the report is genuine, and I have included all my data/-code/designs.

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before re-using any of your previously submitted work for this assessment.

I have not submitted any part of this work for another assessment.

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

My work did not involve human participants, their cells or data, or animals.

UNIVERSITY OF SOUTHAMPTON

ABSTRACT

FACULTY OF ENGINEERING AND PHYSICAL SCIENCES
SCHOOL OF ELECTRONICS AND COMPUTER SCIENCE

A project report submitted for the award of BSc Computer Science

by Lok Hei Lam

This dissertation presents a comprehensive toolbox to interpret the inner dynamics of linear time-varying (LTV) state space models (SSMs), which are a class of sequence models related to classical state space models. This report tackles a major problem in modern deep learning systems: the lack of interpretability, by showing how SSMs can be interpreted in real-world applications.

The SSM is evaluated on various types of datasets, including image, audio, text, and synthetic datasets. We first investigate the model's stability and selectivity by examining the eigenvalue trajectories of the discretised state transition matrix, highlighting how input-dependency modulates system behaviour. Building on this, we analyse sensitivity through the eigenvalue variance and Jacobian-based methods. Finally, we extend the analysis into classical signal processing theory, evaluating the model in the frequency domain, interpreting SSMs as a filter evolving over time.

This report outlines the background, setup, design, implementation, and evaluation of the interpretation methods. The primary objective is to demonstrate that contemporary SSM architectures, such as S4 and Mamba, exhibit meaningful interpretability despite their complexity. Finally, several directions of future research are identified, showing significant opportunities for extending this line of investigation and further advancing the interpretability of deep sequence models.

Contents

1	Introduction	1
1.1	Background	1
1.2	Project Goals and Scope	2
2	Literature Review	5
2.1	Sequence models	5
2.1.1	Markov chain	5
2.1.2	Recurrent Neural Networks	6
2.1.3	LSTM	6
2.1.4	Transformer	7
2.1.5	Efficient attention	8
2.2	State space models	9
2.2.1	Classical state space models	9
2.2.2	Linear time-invariant deep state space models	10
2.2.3	Linear time-variant state space models	12
2.2.3.1	Selective SSM	13
2.2.3.2	Efficient training and inference of Selective SSMs	15
	Parallel Scan	16
	Kernel fusion	17
	Recomputation	17
	Mamba2	18
2.2.4	Mamba	18
2.2.4.1	Constraints on real-valued $\bar{A}$	19
2.2.4.2	Constraints on complex-valued $\bar{A}$	19
2.2.4.3	Real vs. Complex $\bar{A}$	20
2.2.4.4	Discretisation of B	21
2.2.4.5	Initialisation of Δ	21
2.2.4.6	Complex symmetry	21
3	Experimental Setup	23
3.1	Datasets	23
3.1.1	Dataset selection	23
3.1.1.1	CIFAR-10	23
	Training on CIFAR-10	23
3.1.1.2	Speech Commands	25
	Training on Speech Commands	25
3.1.1.3	WikiText-103	26

Tokeniser	26
Training on WikiText-103	27
3.1.1.4 Selective Copying	27
Training on Selective Copying	28
3.1.1.5 Induction Heads	28
Training on Induction Heads	29
3.1.2 Obtaining and storing data	29
3.2 Model training and evaluation	29
3.2.1 Iridis HPC cluster	29
3.2.2 Training results	30
3.3 Mamba implementation	30
3.3.1 Classification architecture	30
3.3.2 Generation architecture	31
4 Methodology	33
4.1 Motivation for interpreting Mamba	33
4.2 Eigenvalue-Based Analysis	34
4.2.1 Analysis of Eigenvalue trajectories	34
4.2.2 Stability Criteria	34
4.3 Sensitivity and temporal dependencies	36
4.3.1 Eigenvalue variance as a temporal metric	37
4.3.2 Jacobian-based analysis	37
4.3.2.1 Efficient computation of Jacobians for Mamba	38
4.4 Spectral analysis	39
4.4.1 Z-transform Formulation	40
4.4.2 Computation of Spectral Responses	40
4.4.3 Complex-valued models	41
4.4.4 Frequency Response Analysis	41
4.4.5 Z-domain Response Analysis	42
5 Evaluation	43
5.1 Results on model stability	43
5.2 Results on eigenvalue analysis on synthetic datasets	45
5.3 Results on Eigenvalue variance	47
5.4 Results on Jacobian-based analysis	49
5.5 Results on Frequency Response	51
5.6 Results on Z-domain response	52
6 Conclusions	55
6.1 Summary	55
6.2 Future works	56
6.3 Reflection	56
A Interpretation plots	59
A.1 Eigenvalue stability plots	60
A.2 Eigenvalue variance plots	61
B Project Management	63

B.1 Project Brief	63
B.2 Gantt Chart	64
Bibliography	67

List of Figures

2.1	Recurrent Neural Network, from Kalita (2025)	6
2.2	LSTM architecture, with forget gate, input gate and output gate, from dida (2025)	7
2.3	Transformer architecture, by Vaswani et al. (2017)	7
2.4	Haar wavelet where $j = 2$ and $k = 0$	13
2.5	Copying (Left), Selective Copying (Top Right), Induction Head (Bottom Right), by Gu and Dao (2023)	13
2.6	S4 Algorithm (Left), S6 Algorithm (Right) B, C, Δ change from shape (D,N) to having dimensions B and L due to input dependency. Details of the parameters can be found on Table 2.1 and Table 2.2, by Gu and Dao (2023)	15
2.7	Softplus function	15
2.8	Parallel scan, where each vertical line is an element of a sequence, from Eppstein (2011)	16
2.9	Overview of the selective SSM with the states (orange) stored and computed in the fast GPU SRAM and the state variables (green) stored in the slow GPU HBM, by Gu and Dao (2023)	18
2.10	A Mamba block, which is repeated N times in a N-layered model. σ is the SiLU activation function (Elfwing et al., 2018), by Hu et al. (2024).	19
2.11	Complex $\bar{A}$ at initialisation	20
3.1	Images from the CIFAR-10 dataset, the classes from top to bottom are: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck	24
3.2	Mamba trained on Wikitext-2: training loss (left), validation loss (right)	26
3.3	Mamba classification model	31
3.4	Mamba generation architecture	32
4.1	Impulse response of an IIR filter	35
4.2	Impulse response of an unstable IIR filters	36
5.1	Eigenvalues of $\bar{A}$ for real-valued Mamba trained on the Speech Commands dataset	43
5.2	Eigenvalues of $\bar{A}$ for complex-valued Mamba trained on the Speech Commands dataset	44
5.3	A slice of the eigenvalues after training on selective copying: $\bar{A}$ (top), $\bar{B}$ (middle) and the input (bottom)	45
5.4	Eigenvalues after training on Induction Head: $\bar{A}$ (top), $\bar{B}$ (middle) and the input (bottom)	46

5.5	The plot of the eigenvalue variance of $\bar{A}$ on Mamba trained on Speech Commands (top). The original Speech Commands signal (bottom)	47
5.6	Eigenvalue variance for Wikitext-103	48
5.7	The Jacobian heatmap for Mamba trained on Speech Commands(top), the input speech signal (bottom), how this plot should be interpreted is described below	49
5.8	Jacobian plot for Mamba trained on Wikitext-103	50
5.9	Frequency response for the dataset Speech Commands	52
5.10	The eigenvalues of $\bar{A}$ (left). The logarithmic Z-transform for Speech Commands, $\log H(z)$, to enhance contrast (right)	53
A.1	Eigenvalues of $\bar{A}$ on Mamba trained on CIFAR-10	60
A.2	Eigenvalues of $\bar{A}$ on Mamba trained on Wikitext-103	60
A.3	Eigenvalues of $\bar{A}$ on Mamba trained on Selective Copying	60
A.4	Eigenvalues of $\bar{A}$ on Mamba trained on Induction Head	60
A.5	Eigenvalues plot of $\bar{A}$ for complex (top row) and real (bottom row) Mamba trained on other datasets	60
A.6	Eigenvalue variance of $\bar{A}$ on Mamba trained on CIFAR-10	61
A.7	Eigenvalue variance of $\bar{A}$ on Mamba trained on selective copying	61
A.8	Eigenvalue variance of $\bar{A}$ on Mamba trained on Induction Head	61
A.9	Eigenvalue variance plot of $\bar{A}$ for other datasets	61
B.1	Gantt chart for Semester 1	65
B.2	Gantt chart for Semester 2	66

List of Tables

2.1	State variables	14
2.2	Dimensions and functions	14
3.1	Hyperparameters used for training Mamba on CIFAR-10.	24
3.2	Hyperparameters used for training Mamba on Speech Commands	25
3.3	Hyperparameters used for training Mamba on WikiText-103	27
3.4	Hyperparameters used for training Mamba on Selective Copying	28
3.5	Training results across tasks	30
4.1	Time efficiency comparison to calculate the Jacobian matrix for the Speech Commands dataset with $T = 16000$	39

Chapter 1

Introduction

1.1 Background

State space models (SSMs) are a fundamental framework originated from classical control theory that models a dynamic system over time using state variables. Recently, deep SSMs architectures are developed to model sequential data. These models have shown strong performances and are capable of modelling very long sequences. They are a strong contender for fundamental models (FMs), alternative to Transformers ([Vaswani et al., 2017](#)).

Linear time-invariant (LTI) structured state space sequence models such as S4 ([Gu et al., 2021](#)) and S5 ([Smith et al., 2022](#)) have demonstrated their ability to model long sequences by constraining their kernel structures, beating state of the art transformer models in Long Range Arena ([Tay et al., 2020](#)) and speech recognition. They have the advantages of transformer and RNN in terms of fast training and inference by switching between convolution representation and recurrent representation. However, these LTI models lack of selectivity makes them unable to match transformers' performance on content-based tasks such as language modelling.

Mamba ([Gu and Dao, 2023](#)) improves these models by removing the constraint of time-invariance, making the state variables input-dependent, causing these new models to be linear time-variant (LTV). The input dependency introduced a selection mechanism to the model, giving it the ability to select relevant information from the input, allowing it to choose to remember or ignore the input. Although this has prevented the use of convolution during training, a hardware-aware parallel algorithm is used instead for efficient training and inference while still having linear scaling with sequence length during inference.

Although these new models have been widely successful in various areas, their inner dynamics are still unclear. In order to apply these models in critical systems, it is important to understand what the models have been learning to determine the explainability and trustworthiness of the models. In this research, we will investigate on the interpretability of Mamba, developing techniques to deepen our understanding of these architectures.

1.2 Project Goals and Scope

This research investigates the research question: *What did these linear time-variant (LTV) state space models learn?* This question addresses the current literature gap regarding to the interpretation of these SSMs. The interpretation these models is important to determine the safeness of these models and to access their potential in becoming the new state-of-the-art fundamental model.

This project’s main goal is to explore different interpretation techniques and propose a theoretical framework in analysing LTV SSMs. This research will focus on evaluating SSMs in the following areas:

1. Eigenvalue-based analysis
2. Sensitivity analysis
3. Spectral analysis

The *scope* of the project focuses solely on interpreting the LTV SSM Mamba. Experiments of the project will primarily be conducted on the foundational Mamba block based on the GitHub repository by [Ran-Milo et al. \(2024\)](#), other models such as Vision Mamba ([Zhu et al., 2024](#)) and U-Mamba ([Ma et al., 2024](#)) won’t be used as the aim of this research is to investigate the core mechanism of Mamba. The project prioritises in developing a theoretical framework for understanding LTV SSMs and strategies in interpreting these models. We are only interested in interpretation methods specific to state space models, hence we will pay no attention to any post-hoc and model-agnostic methods such as SHAP ([Lundberg and Lee, 2017](#)) and LIME ([Ribeiro et al., 2016](#)). The research will also not attempt to investigate methods to either improve the models on a specific evaluation metrics, propose new architecture or improve the efficiency of the models.

In addition, this research is primarily targeted at academics studying model transparency and safety, as well as practitioners in machine learning seeking a deeper understanding of the interpretability of Mamba for practical deployment. The work are made to in a

manner to support broader accessibility within the research community by introducing the audience to linear systems, deep learning and signal processing concepts.

Chapter 2

Literature Review

2.1 Sequence models

Sequential modelling lies at the heart of machine learning systems that must reason about information over time, this includes language, speech, images, DNA, stock market and more. This section will be a brief review of the history in sequential modelling for us to understand to origin and the role of state space models.

2.1.1 Markov chain

Early sequential modelling in machine learning uses methods rooted in statistics and probability. The Markov chain or Markov models models a sequence of dependent events as a chain of probabilistic transitions. A Markov chain assumes that the future state only depends on the current state, so that we can form a Markov chain:

$$P(x) \approx P(x_k|x_{k-1})P(x_{k-1}|x_{k-2}) \dots P(x_2|x_1)P(x_1) \quad (2.1)$$

This property of the future only depending on the present but not the past makes the mathematical model more tractable and laid the groundwork for sequential modelling. This frame work is powerful enough to model simple sequences such as next character prediction and simple biological sequences. However, the memoryless property of Markov chains makes it suffer from long sequences as the future state might also depend on the past, like languages.

2.1.2 Recurrent Neural Networks

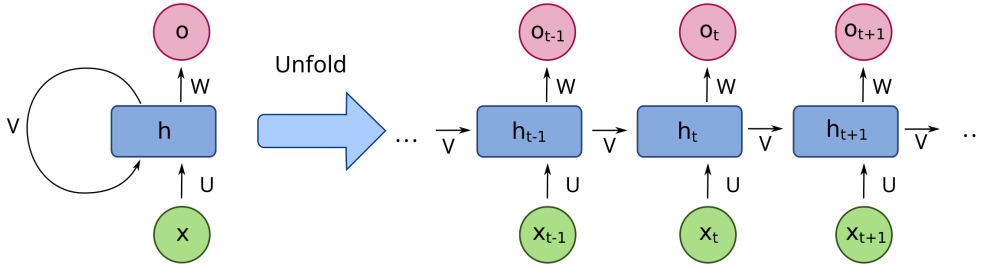


FIGURE 2.1: Recurrent Neural Network, from [Kalita \(2025\)](#)

RNNs are neural networks that contain feedback connections that allows the network the maintain a hidden state that evolves over time, similar to modern SSMs, see [Figure 2.1](#). Early RNNs faced serious limitations such as exploding and vanishing gradient and their limited memory over a relatively short period of time. Making them impossible to train on arbitrarily long sequences.

2.1.3 LSTM

Long Short-Term Memory networks (LSTM) ([Hochreiter and Schmidhuber, 1997](#)) are a class of RNN-based sequence models that solves the problem of exploding and vanishing gradients, see [Figure 2.2](#). It still retains the hidden state representation of RNNs but introduces three learnable gates. The gates include the forget gate, input gate and the output gate. These gates combined effect made this architecture the first sequence model that allows the learning of long-range dependencies and revolutionised sequence modelling. This has applications in speech recognition, language modelling, time-series prediction and more.

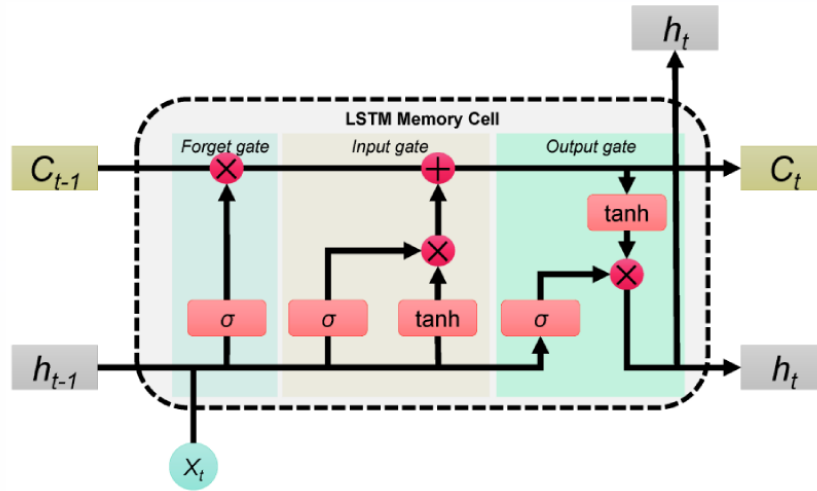


FIGURE 2.2: LSTM architecture, with forget gate, input gate and output gate, from [dida \(2025\)](#)

2.1.4 Transformer

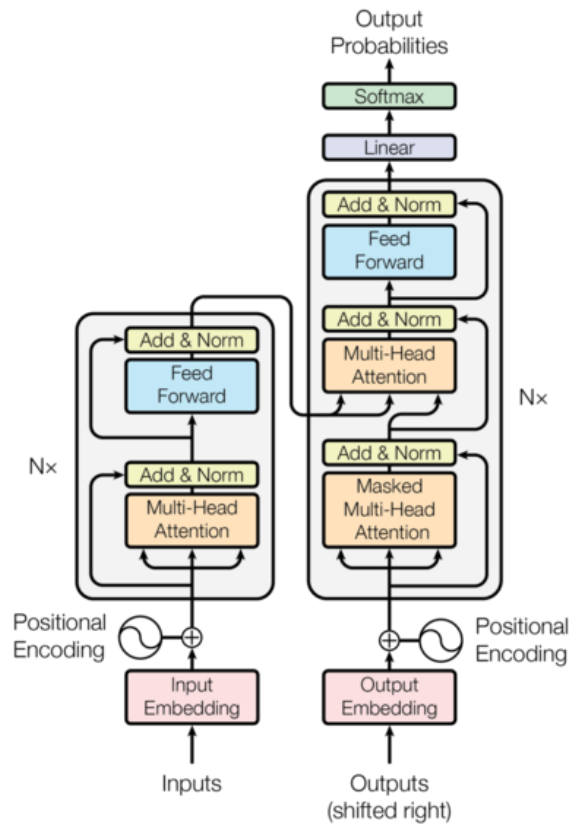


FIGURE 2.3: Transformer architecture, by [Vaswani et al. \(2017\)](#)

Even though LSTM handles long-range dependencies, they still process sequences sequentially and compressed all information into a hidden state. This stops the RNN-based architectures to be trained on modern GPUs in parallel which limited their applications on large scale data.

Bahdanau et al. (2014) introduced the modern attention mechanism, initially for machine translation, allowing the model to attend to different parts of the input sequence as needed. It enables direct access to any timestep and a longer context window. Vaswani et al. (2017) improved upon this and introduced the transformer architecture which completely removes recurrence and replace it with self-attention. The self attention mechanism can be summarised by this equation:

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.2)$$

In a self-attention head, each token has a query Q , key K and value V vector. It computes the similarity between each query and key using a dot product QK^T and softmax turns the similarity scores into a probability distribution, and finally the weighted sum of values produces the final representation incorporating information from others based on relevance.

The abandoning of recurrence allows transformers to be trained in parallel with much greater scalability. It has quickly surpassed LSTM in various fields and are used in the most popular large language models like the GPT (Brown et al., 2020) and Llama (Touvron et al., 2023).

2.1.5 Efficient attention

The success of Transformer architectures has largely been driven by the self-attention mechanism, but a key limitation is its quadratic time and memory complexity with respect to sequence length, making it difficult to scale to long sequences.

To address this issue, several approaches have been proposed. Sparse attention restricts attention to local or structured patterns (Child et al., 2019), while Linformer reduces the dimensionality of the attention matrix with low-rank approximations (Katharopoulos et al., 2020). There are also attempts at making attention scale linearly, such as Linear Transformers (Katharopoulos et al., 2020) and Performer (Choromanski et al., 2022).

While these methods improve computational efficiency, they often introduce tradeoffs in various areas. This motivates an alternative line of research in using state space models (SSMs) for sequence modelling that comes from classical control theory (Gu et al., 2021). These models are related to classical RNNs, offering linear complexity, meanwhile

demonstrating competitive performance in many benchmarks. This transition from attention-based models to SSMs provides the foundation for the next section.

2.2 State space models

2.2.1 Classical state space models

State space models are originated from control theory. It is a mathematical representation of a physical system using state variables to model the system's behaviour over continuous time through first-order differential equations. A general state-space representation with state variables $A(t), B(t), C(t), D(t)$ is written as:

$$\begin{aligned}\frac{dy}{dx} &= A(t)x(t) + B(t)u(t) \\ y(t) &= C(t)x(t) + D(t)u(t)\end{aligned}\tag{2.3}$$

$x(t)$ here can be viewed as the hidden state of the model and A is the state matrix, it determines how state evolves over time.

For a linear time-invariant system, the state variables are constant overtime instead of a function of t . This allows us to write the solution to [Equation 2.3](#) with τ as the integration dummy variable as:

$$x(t) = e^{A(t-t_0)}x(t_0) + \int_{t_0}^t e^{A(t-\tau)}Bu(\tau)d\tau\tag{2.4}$$

To solve for a linear time-varying system, where the state variables are a function of t , a state transition matrix Φ is defined as:

$$\dot{\Phi}(t, \tau) = A(t)\Phi(t, \tau) \quad \Phi(\tau, \tau) = I\tag{2.5}$$

$\Phi(t, \tau)$ is defined as the state transition matrix at time t if τ is the initial time. Φ has the following properties:

$$\begin{aligned}x(t) &= \Phi(t, t_0)x_0 \\ \Phi(t, t_0) &= \Phi(t, t_1)\Phi(t_1, t_0) \\ \Phi^{-1}(t, t_0) &= \Phi(t_0, t)\end{aligned}\tag{2.6}$$

And the solution to the linear time-varying system from Equation 2.3 with τ as an integration dummy variable is (Dahleh et al.):

$$x(t) = \Phi(t, t_0)x(t_0) + \int_{t_0}^t \Phi(t, \tau)B(\tau)u(\tau)d\tau \quad (2.7)$$

State space models are applied in various fields such as electrical engineering, economics and neuroscience. They have proven to be a great model for all dynamic systems, and give a flexible and controllable framework to model the physical world.

2.2.2 Linear time-invariant deep state space models

Linear time-invariant deep state space models are a class of deep learning architecture developed to model sequential data that have a deep connection to CNNs and RNNs. In this dissertation, the terms "SSM" and "state space model" are used to refer exclusively to structured state space model or S4 (Gu et al., 2021). The state space model is defined by the equation:

$$\begin{aligned} x'(t) &= Ax(t) + Bu(t) \\ y(t) &= Cx(t) + Du(t) \end{aligned} \quad (2.8)$$

In order to apply this to a discrete input sequences, the SSM has to be discretised using a time step variable Δ . In this context, Δ represents the time resolution of the input, the system is discretised by the following using the zero-order hold (ZOH) (Tóth et al., 2008):

$$\bar{A} = \exp(\Delta A) \quad \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B \quad (2.9)$$

The new discretised system is described by:

$$\begin{aligned} x_k &= \bar{A}x_{k-1} + \bar{B}u_k \\ y_k &= \bar{C}x_k + \bar{D}u_k \quad \bar{C} = C \quad \bar{D} = D \end{aligned} \quad (2.10)$$

Equation (2.10) depicts a linear time-invariant (LTI) system as the state variables are constant over the sequence length. $\bar{D}$ can be viewed as a skip connection and it will be omitted from here on. The SSM is now a recurrence of x_k , which allows it to be computed as a recurrent neural network (RNN) which are often represented by (Schmidt, 2019):

$$\begin{aligned} H_t &= \phi_h(X_t W_{xh} + H_{t-1} W_{hh} + b_h) \\ O_t &= \phi_o(H W_{ho} + b_o) \end{aligned} \quad (2.11)$$

In equation (2.11), we can see the relationship between SSM and RNN. The x_k from SSM can be treated as the hidden state H_t from the RNN and the input u_k is equivalent to X_t from the RNN. The only difference between these two is the lack of bias terms and non-linear activation function ϕ in the SSM. This allows the SSM overcome the notoriously inefficient training of RNN by utilising a convolution representation:

$$\begin{aligned} y_k &= \overline{CA}^k \bar{B}u_0 + \overline{CA}^{k-1} \bar{B}u_1 + \cdots + \overline{CA} \bar{B}u_{k-1} + \bar{B}u_k \\ y &= \bar{K} * u \end{aligned} \quad (2.12)$$

$$\bar{K} \in \mathbb{R}^L := \bar{K} = (\overline{CB}, \overline{CAB}, \dots, \overline{CA}^{L-1} \bar{B}) \quad (2.13)$$

Equation (2.12) is a simple convolution and can be computed efficiently using the Fast Fourier Transform (FFT) algorithm. This is due to the relationship given by the Convolution Theorem (Lyons, 2011a):

$$x(t) * h(t) = \mathcal{F}^{-1}[\mathcal{F}\{x(t)\} \cdot \mathcal{F}\{h(t)\}] \quad (2.14)$$

This states that convolution in the time domain becomes multiplication in the frequency domain, which is significantly cheaper computationally than convolutions. In other words, if we take the Fourier Transform of the kernel from (2.13) and the input, multiply them and take the Inverse Fourier Transform of the result, we get the convolution of the kernel and the input. The FFT algorithm is a well-known algorithm for computing Fourier Transform/Inverse Fourier Transform quickly, in $O(n \log n)$ time, where n is the sequence length.

In order to capture long range dependencies, LTI SSMs like S4 initialise the matrix $A \in \mathbb{R}^{N \times N}$ using the HiPPO theory of memorisation in continuous time (Gu et al., 2020). This special matrix is initialised by:

$$A_{nk} = - \begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n > k \\ n+1 & \text{if } n = k \\ 0 & \text{if } n < k \end{cases} \quad (2.15)$$

However, the cost of computing $\bar{K}$ from (2.13) is expensive. This is mainly due to repeated matrix multiplication of $\bar{A}$, requiring $O(N^2L)$ operations and NL space. If we can diagonalise A , we will be able to compute the kernel in $O((N+L)\log^2(N+L))$

operations as a Vandermonde product. However, A cannot be naively diagonalised as entries of a diagonalised HiPPO matrix grows exponentially with its dimension N . In order to compute the SSM convolution kernel efficiently, Gu et.al have demonstrated that the HiPPO matrix can be decomposed into diagonal plus low-rank (DPLR) form with:

$$\begin{aligned} A &= \Lambda - PQ^* \\ \Lambda &\in \mathbb{C}^{N \times N} \quad P, Q \in \mathbb{C}^{N \times 1} \end{aligned} \quad (2.16)$$

This results in $\tilde{O}(N + L)$ operations and $O(N + L)$ space to compute the convolution filter $\tilde{K}$.

The resulting structured state space model or S4 is an efficient solution for sequential modelling. It outperforms transformer in Long Range Arena (LRA) (Tay et al., 2020) and in sequential benchmarks such as Speech Commands classification (SC10), sCIFAR-10 classification and WikiText-103 language modeling.

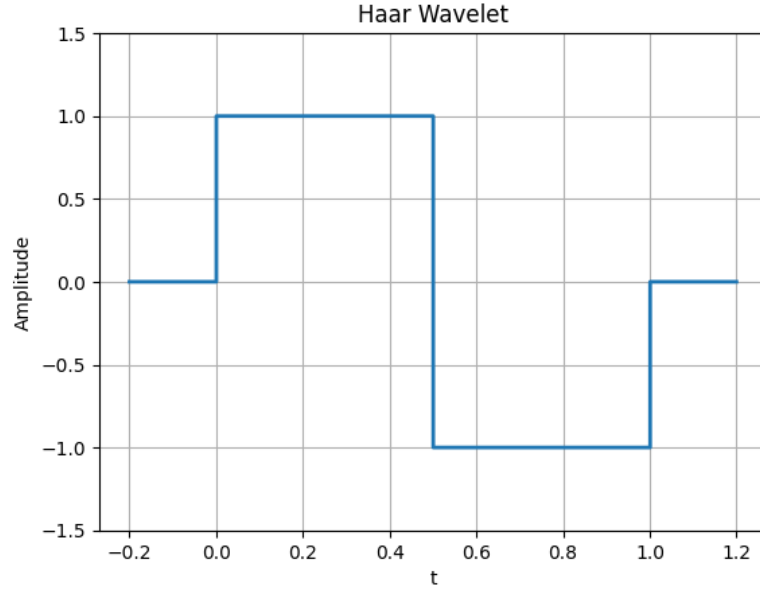
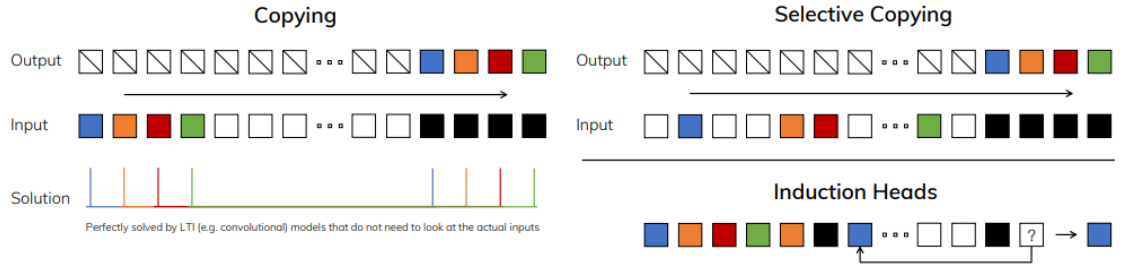
2.2.3 Linear time-variant state space models

Although SSMs are capable of efficiently modelling long sequential data, their lack of selectivity makes them unable to recall and compare tokens across the sequence. Fu et al. (2022) have demonstrated that S4 is still performing worse than transformer in language modelling. This can be shown in their poor performance in the synthetic tasks selective copying (Arjovsky et al., 2015) and induction heads (Olsson et al., 2022), which are related to the ability of the model in recalling tokens (Figure 2.5). This has motivated the removal of the linear time-invariance constraint and introduce input dependency to the state variables in Mamba.

Huang et al. (2025) have demonstrated that Mamba can approximate Haar Wavelets but LTI systems can only represent Fourier bases which are essentially just sine and cosine waves, refer to Equation 2.4. Haar wavelets are wavelets of the form:

$$\psi_{n,k}(t) = \begin{cases} 2^{j/2} & \text{if } \frac{k}{2^j} \leq t < \frac{k+0.5}{2^j} \\ -2^{j/2} & \text{if } \frac{k+0.5}{2^j} \leq t < \frac{k+1}{2^j} \\ 0 & \text{otherwise} \end{cases} \quad (2.17)$$

An example of this can be seen in Figure 2.4, with j and k as the scaling and shifting factor respectively and they are discontinuous. This proves that Mamba is able to represent discontinuous functions effectively, whereas LTI SSMs like S4 cannot approximate

FIGURE 2.4: Haar wavelet where $j = 2$ and $k = 0$ FIGURE 2.5: Copying (Left), Selective Copying (Top Right), Induction Head (Bottom Right), by [Gu and Dao \(2023\)](#)

discontinuous functions as effectively as Fourier bases are generally worse than wavelets in modelling discontinuous functions.

This shows that Mamba can be treated as a model for discrete signals, that is why it outperforms S4 in tasks such as texts and DNA modelling which are discrete data with dense information and requires content-aware reasoning. Although Mamba overcomes the weaknesses of LTI SSM on discrete data, they perform worse than S4 in continuous-time data such as images and audio, this will be shown in later experiments.

2.2.3.1 Selective SSM

Selective SSM/S6 was introduced to Mamba to model information-dense sequential data that requires selectivity, such as languages ([Gu and Dao, 2023](#)). It introduces input dependency to the model's (B, C, Δ) parameters:

Parameter	Shape	Description
$\mathbf{x}$	(B, L, D)	Input sequence
$\mathbf{y}$	(B, L, D)	Output sequence
$\mathbf{A}$	(D, N)	The state-transition matrix, initialised with the HiPPO theory, using the S4D-real matrix, where $A_n = -(n + 1)$ (Gu et al., 2022). A is a diagonal $N \times N$ matrix, hence it only requires N entries to represent it. It has dimension (D, N) as it treats each input dimension D independently.
$\mathbf{B}$	(B, L, N)	The input projection matrix. It is computed by this input-dependent equation: $B = s_B(x) = \text{Linear}_N(x)$
$\mathbf{C}$	(B, L, N)	The output projection matrix. It is computed by this input-dependent equation: $C = s_C(x) = \text{Linear}_N(x)$
Δ	(B, L, D)	Δ is computed by the equation: $\tau_\Delta(\text{Parameter} + s_\Delta(x))$, $s_\Delta(x) = \text{Broadcast}_D(\text{Linear}_1(x))$, and $\tau_\Delta = \text{softplus}$. From the equation, $s_\Delta(x)$ broadcasts a scalar to D channel dimension for a given time step, then softplus is applied with bias (Parameter) added.
$\bar{\mathbf{A}}$	(B, L, D, N)	The discretised $\mathbf{A}$, by Equation 2.9, this makes the $\bar{\mathbf{A}}$ input dependent. The dimension (B, L, D, N) means that for each sample, each token with dimension D has a N -dimensional vector projection for N state dimensions.
$\bar{\mathbf{B}}$	(B, L, D, N)	The discretised $\mathbf{B}$, by Equation 2.9. The dimensions are the same as $\bar{\mathbf{A}}$.

TABLE 2.1: State variables

Parameter	Description
$\mathbf{B}$	Batch size
$\mathbf{L}$	Sequence length
$\mathbf{D}$	Input channel dimension
$\mathbf{N}$	Hidden state dimension
$\text{Linear}_N(\mathbf{x})$	A fully-connected linear layer in a neural network that outputs N features with no activation function that takes the sequence x as the input.
$\text{Broadcast}_D(\mathbf{x})$	Repeats an input x across D dimensions.
softplus	The softplus function: $f(x) = \ln(1 + e^x)$, it is a smooth approximation to the ReLU function, see Figure 2.7.

TABLE 2.2: Dimensions and functions

Algorithm 1 SSM (S4)	Algorithm 2 SSM + Selection (S6)
Input: $x : (B, L, D)$ Output: $y : (B, L, D)$ 1: $\mathbf{A} : (D, N) \leftarrow \text{Parameter}$ $\triangleright$ Represents structured $N \times N$ matrix 2: $\mathbf{B} : (D, N) \leftarrow \text{Parameter}$ 3: $\mathbf{C} : (D, N) \leftarrow \text{Parameter}$ 4: $\Delta : (D) \leftarrow \tau_\Delta(\text{Parameter})$ 5: $\overline{\mathbf{A}}, \overline{\mathbf{B}} : (D, N) \leftarrow \text{discretize}(\Delta, \mathbf{A}, \mathbf{B})$ 6: $y \leftarrow \text{SSM}(\overline{\mathbf{A}}, \overline{\mathbf{B}}, \mathbf{C})(x)$ $\triangleright$ Time-invariant: recurrence or convolution 7: return y	Input: $x : (B, L, D)$ Output: $y : (B, L, D)$ 1: $\mathbf{A} : (D, N) \leftarrow \text{Parameter}$ $\triangleright$ Represents structured $N \times N$ matrix 2: $\mathbf{B} : (B, L, N) \leftarrow s_B(x)$ 3: $\mathbf{C} : (B, L, N) \leftarrow s_C(x)$ 4: $\Delta : (B, L, D) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$ 5: $\overline{\mathbf{A}}, \overline{\mathbf{B}} : (B, L, D, N) \leftarrow \text{discretize}(\Delta, \mathbf{A}, \mathbf{B})$ 6: $y \leftarrow \text{SSM}(\overline{\mathbf{A}}, \overline{\mathbf{B}}, \mathbf{C})(x)$ $\triangleright$ Time-varying: recurrence (<i>scan</i>) only 7: return y

FIGURE 2.6: S4 Algorithm (Left), S6 Algorithm (Right) B, C, Δ change from shape (D, N) to having dimensions B and L due to input dependency. Details of the parameters can be found on Table 2.1 and Table 2.2, by Gu and Dao (2023)

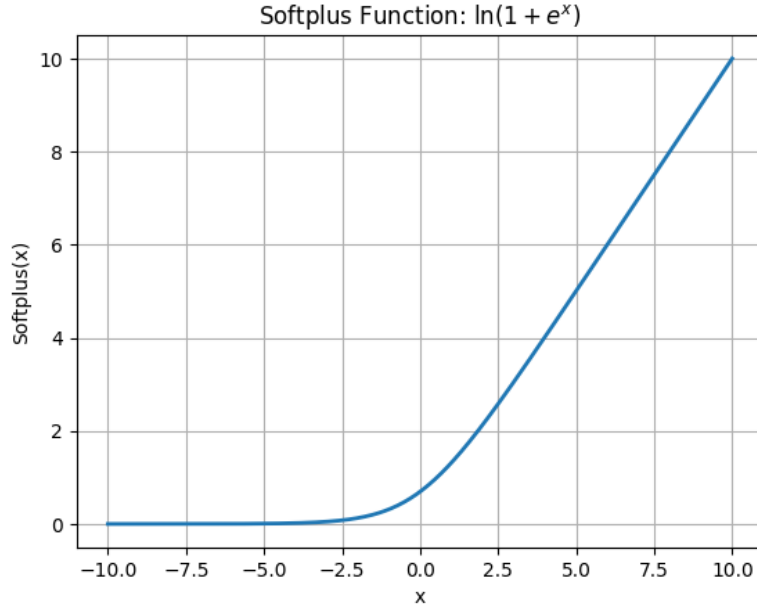


FIGURE 2.7: Softplus function

2.2.3.2 Efficient training and inference of Selective SSMs

Since S6 introduced linear time-variance, it cannot switch to a convolution representation as discrete convolutions are only defined for linear time-invariant systems (Oppenheim et al., 1996). This means that it has to seek another way to compute the recurrence efficiently.

Gu and Dao (2023) introduced a hardware-aware algorithm, which includes three techniques: **parallel scan**, **kernel fusion** and **recomputation**. This is achieved by utilising properties of GPUs more efficiently in their memory hierarchy, using ideas from FlashAttention (Dao et al., 2022), and a hardware-aware scan operation.

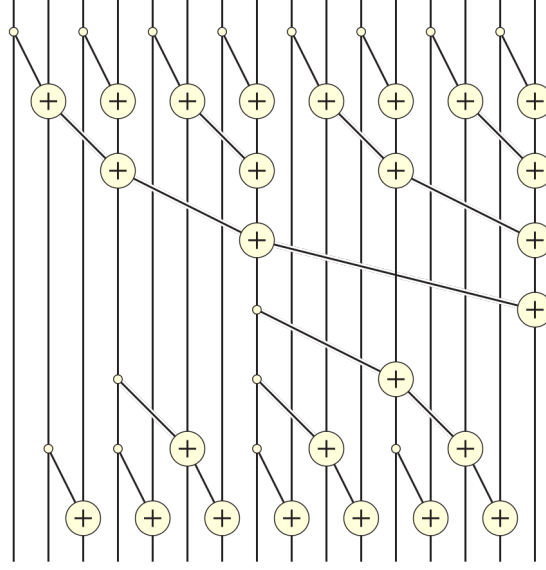


FIGURE 2.8: Parallel scan, where each vertical line is an element of a sequence, from [Eppstein \(2011\)](#)

Parallel Scan The recurrent representation is linear and slow to compute as x_k depends on the previous state x_{k-1} , so it cannot utilise modern GPUs to parallelise the operation. The problem of recurrent computation is highly related to a problem called prefix sums ([Blelloch; Smith et al., 2022](#)), which is formulated like this: The prefix sums take in a sequence with n elements

$$[a_0, a_1, \dots, a_{n-1}]$$

and returns a sequence with each element as the sum of all of its prefixes

$$[a_0, a_0 + a_1, \dots, a_0 + a_1 + \dots + a_{n-1}]$$

The relationship between prefix sums and SSMs can be viewed as follows:

For prefix sums:

$$a_k = \sum_{n=0}^k a_n$$

For SSMs:

$$y_k = \sum_{n=0}^k \overline{CA}^n \bar{B} u_{k-n}$$

The core mechanism of parallel scan is the exploitation of associativity of the add operation, where $(a + b) + c = a + (b + c)$. Although the computation is inherently sequential, computations of future elements that do not require the previous states can be computed

in parallel. This can be demonstrated with this example:

Let input $u = [u_0, u_1, u_2, u_3]$ and state $x_0 = 0$

First sweep:

$$x_1 = \bar{A}x_0 + \bar{B}u_1 \quad x_3 = \bar{A}x_2 + \bar{B}u_3 = \bar{A}^2x_1 + \bar{A}\bar{B}u_2 + \bar{B}u_3$$

In the first sweep, the value of x_1 and $\bar{A}\bar{B}u_2 + \bar{B}u_3$ from x_3 can be computed in parallel.

Second sweep:

$$x_2 = \bar{A}x_1 + \bar{B}u_2$$

In the second sweep, the value of x_2 and x_3 can be computed from the value of x_1 in parallel.

Final sweep:

$$x_4 = \bar{A}x_3 + \bar{B}u_4$$

In the final sweep, x_4 is computed with x_3 .

The operation takes parallel time $O(T_{\odot} \log L)$, assuming there is at least L processors, where $T_{\odot}$ is the matrix multiplication cost ([Smith et al., 2022](#)).

Kernel fusion Most deep learning architectures are trained on highly parallelised units called GPUs. Although they are very efficient in floating point calculations and matrix multiplications, they are heavily memory-bound ([Ivanov et al., 2020](#)).

GPU memory is split into the slow but larger GPU high-bandwidth memory (HBM) and the fast but smaller GPU SRAM. Kernel fusion fuses the discretization step, the parallel scan, and the multiplication with C into one kernel and performs them all in the fast SRAM; the result is written in HBM, see [Figure 2.9](#). This reduces the number of IO operations by a factor of $O(N)$, where N is the state dimension and speeds up the operation by 20-40 times ([Gu and Dao, 2023](#)).

Recomputation When an arbitrarily long sequence is used as an input, the memory storing the intermediate states of size (B, L, D, N) may blow up. Recomputation reduces the memory cost by not saving the intermediate states. During the backward

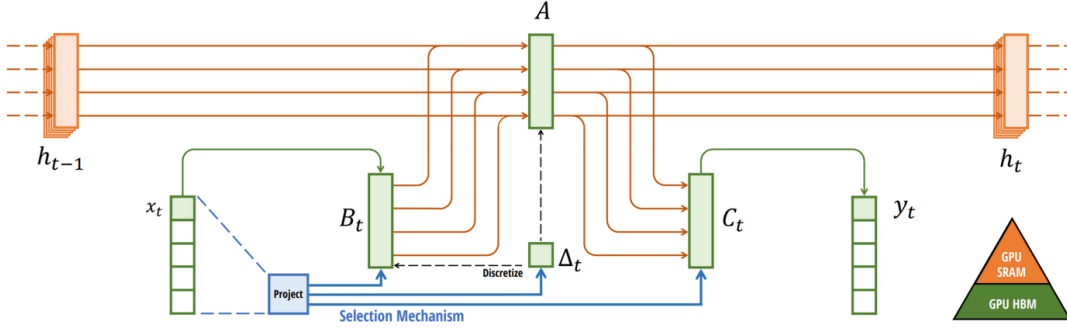


FIGURE 2.9: Overview of the selective SSM with the states (orange) stored and computed in the fast GPU SRAM and the state variables (green) stored in the slow GPU HBM, by [Gu and Dao \(2023\)](#)

pass, the intermediate states are recomputed using the inputs (Δ, A, B, C) to calculate the gradients. Since reading intermediate states from HBM requires $O(BLDN)$ reads; this actually speeds up the training process by reducing the number of IO operations. As a result, the S6 layer has the same memory requirement as FlashAttention ([Dao et al., 2022](#)).

Mamba2 [Dao and Gu \(2024\)](#) proposed Mamba2, which further improves these ideas and invented the structured state space duality (SSD) framework. This builds a connection between attention ([Vaswani et al., 2017](#)) and SSMs. Since a lot of community effort has been put into improving the efficiency of transformers. This allows SSMs to utilise the efficient algorithms designed for transformers in SSMs.

2.2.4 Mamba

Mamba is an SSM architecture that contains the core S6 layer, a 1D convolution layer and linear projections, which are fully connected perceptron layers ([Figure 2.10](#)). The Mamba block is repeated and interleaved with layer normalisation ([Ba et al., 2016](#)) and residual connections ([He et al., 2015a](#)).

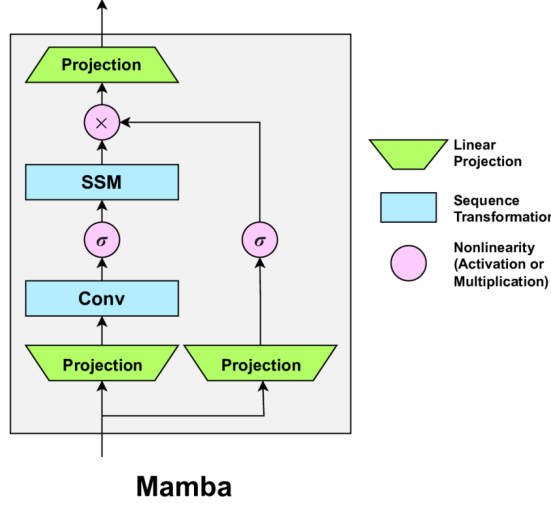


FIGURE 2.10: A Mamba block, which is repeated N times in a N -layered model. σ is the SiLU activation function (Elfwing et al., 2018), by Hu et al. (2024).

2.2.4.1 Constraints on real-valued $\bar{A}$

In the official implementation of Mamba from the authors of the paper (Gu and Dao, 2023), after they have initialised the diagonal continuous time A matrix as:

$$A_n = -(n + 1) \quad (2.18)$$

They compute $A_{\log} = \ln(-A)$ so that $A = -e^{A_{\log}}$, and $A_{\log}$ is updated during back propagation. This ensures A is always negative, as the $-e^x$ is always negative. The ZOH discretised $\bar{A}$ is constrained by:

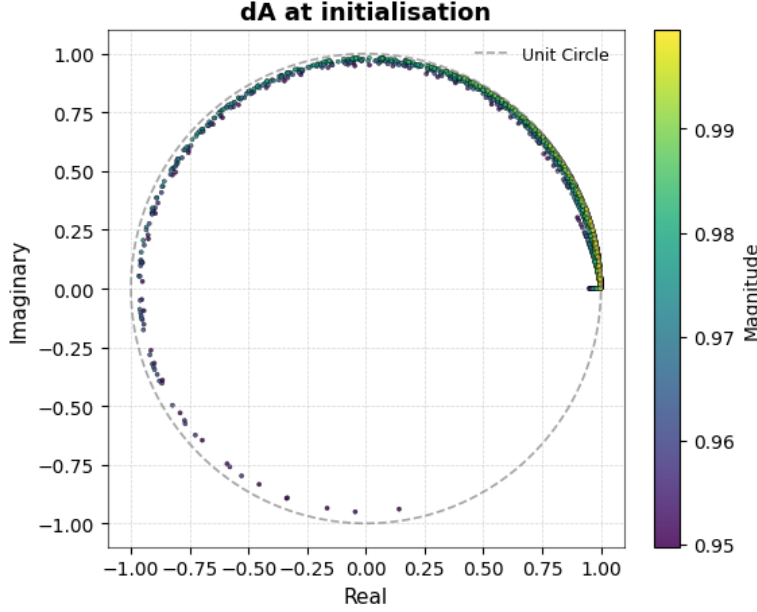
$$\begin{aligned} \Delta &= \text{softplus}(\text{Linear}(x)) \quad \Delta > 0 \\ \bar{A} &= e^{\Delta A} \quad \Delta A < 0 \\ |\bar{A}| &< 1 \end{aligned} \quad (2.19)$$

This constrains the model's eigenvalues to be between 0 and 1, and this is important for model stability to avoid exploding/vanishing gradient. This will be discussed more later.

2.2.4.2 Constraints on complex-valued $\bar{A}$

The complex-valued diagonal A matrix is initialised as:

$$A_n = -\frac{1}{2} + i\pi n \quad (2.20)$$

FIGURE 2.11: Complex $\bar{A}$ at initialisation

This initialisation comes from [Gu et al. \(2022\)](#). The constraint follows the same pattern as the real-valued matrix. The real part of A is $A_{\log}$, and the real and imaginary parts are stored as separate parameters of the model that are used during back propagation. The real part is calculated by $-A_{\log}$, so it is always negative. The constraint on the ZOH complex $\bar{A}$ is as follows:

let $A_n = -a + i\pi b$ where a and b are real numbers after some epochs and $a > 0$

$$\bar{A} = e^{\Delta A} = e^{-a\Delta} e^{i\pi b\Delta}$$

$$\Delta > 0 \quad |e^{-a\Delta}| < 1$$

$$e^{i\pi b\Delta} = \cos(\pi b\Delta) + i \sin(\pi b\Delta)$$

$$|e^{i\pi b\Delta}| = \cos^2(\pi b\Delta) + \sin^2(\pi b\Delta) = 1$$

$$|\bar{A}| = |e^{-a\Delta}| |e^{i\pi b\Delta}| = |e^{-a\Delta}|$$

$$|\bar{A}| < 1$$

(2.21)

This shows that the eigenvalues of $\bar{A}$ lie within the unit circle of the complex plane, for similar reasons to the real-valued $\bar{A}$.

2.2.4.3 Real vs. Complex $\bar{A}$

SSMs developed from before S6 primarily used complex-valued state space to model sequences ([Gu et al., 2021, 2022](#)). However, real-valued S6 performs as good as complex-valued ones or even better in some tasks such as language modelling ([Gu and Dao, 2023](#)),

but in tasks such as audio complex-valued S6 still performs better than real-valued S6.

2.2.4.4 Discretisation of B

Instead of using Equation 2.9 to discretise B , the implementation uses an approximation using Euler’s forward method, where:

$$\begin{aligned} x(t + \Delta) &\approx (I + A\Delta)x(t) + B\Delta u(t) \\ \bar{B} &\approx B\Delta \end{aligned} \tag{2.22}$$

This simplifies the implementation without affecting the performance.

2.2.4.5 Initialisation of Δ

In the implementation, they initialised a `dt_bias` term that corresponds to the Parameter of Δ in Figure 2.6 as the inverse softplus of values between `dt_max` and `dt_min`. Assuming the mean of $\text{Linear}(x)$ is zero on initialisation (He et al., 2015b), the Δ values are approximately $\text{softplus}(\Delta_{\text{bias}})$, which is between `dt_max` and `dt_min`. The values of `dt_bias` are usually initialised between 0.001 and 0.1.

2.2.4.6 Complex symmetry

In classical signal processing, for the output of a complex-parametrised system to have real-valued output, its matrices’ values should come in conjugate pairs ($a + ib$ and $a - ib$). Instead of storing both the state and its conjugate explicitly, which will double the state size, we only store one of each pair, take the real part, and multiply the output by 2 to recover the correct real-valued output magnitude.

Chapter 3

Experimental Setup

3.1 Datasets

3.1.1 Dataset selection

Since this research investigates the interpretability of Mamba, we selected a variety of widely available and benchmarked datasets to ensure our results are robust and generalisable. This includes a selection of image, audio, text, and synthetic datasets. Since mamba is a sequential model, the datasets will mainly contain sequential datasets.

3.1.1.1 CIFAR-10

CIFAR-10 contains 60,000 32×32 low-resolution and labelled animals or motorised vehicle images from 10 different classes designed for image classification. This dataset is chosen because it contains no human data and is publicly available thanks to [Krizhevsky \(2009\)](#). CIFAR-10 provides a well-balanced and widely studied benchmark for evaluating model behaviour in image classification tasks. CIFAR-10 is also a subset of the CIFAR-100 dataset that contains 100 different classes. However, CIFAR-100 contains human data and is too excessive for our task, so it is not chosen for this project.

Training on CIFAR-10 Since images are inherently two-dimensional and Mamba is a sequential model, the images need to be serialised before feeding them into the model. The images originally have dimensions $32 \times 32 \times 3$, the last dimension is the colour channels. They are then flattened into a 1024×3 tensor, as $32 \times 32 = 1024$. The tensor is then fed into the model pixel by pixel, and the model outputs the final logit

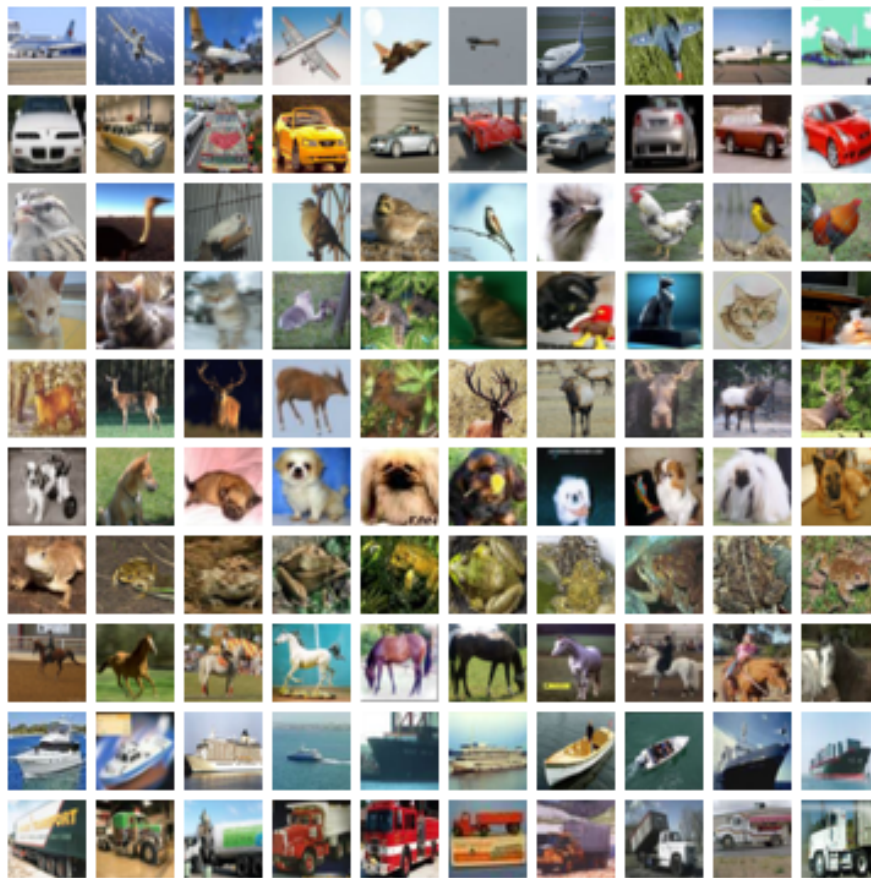


FIGURE 3.1: Images from the CIFAR-10 dataset, the classes from top to bottom are: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck

vector of dimension 1×10 since there are 10 classes.

Hyperparameters:

Parameter	Value
Learning rate	10^{-4}
Batch size	32
Epochs	32
Weight decay	0.1
Model dimension	128
State dimension	32
Number of layers	3
Optimizer	Adam
Loss function	Cross-entropy

TABLE 3.1: Hyperparameters used for training Mamba on CIFAR-10.

3.1.1.2 Speech Commands

The Speech Commands dataset by [Warden \(2018\)](#) is a collection of short, single-word audio recordings to train machine learning models on keyword spotting tasks in speech recognition. Each recording corresponds to a labelled spoken command, including common terms such as "yes" and "left", is a 1-second clip sampled at 16 kHz. We have chosen this dataset because we are interested in how Mamba models time series data and its relationship to traditional signal processing filters. We chose a subset of commands, which includes: yes, no, up, down, left, right, on, off, stop, and go. This is because this selection provides a more manageable yet representative set of classes, and since audio data are relatively lengthy, this allows the model to converge more quickly and learn discriminative acoustic features more effectively. Another reason is that we aim to interpret the model, but not apply the model in real-world tasks, so it is unnecessary to use the entire dataset.

Training on Speech Commands Each audio clip is a 1×16000 dimensional vector, since they are all 1-second mono audio sampled at 16 kHz. The vectors are fed into the model sequentially, one sample at a time, and the final output is a vector of dimension 1×10 with 10 classes. A gradient clipping of 1.0 is necessary as audio clips are especially long and tend to lead to exploding gradients during training.

Hyperparameters:

Parameter	Value
Learning rate	0.001
Batch size	16
Epochs	20
Weight decay	0.1
Model dimension	64
State dimension	32
Number of layers	3
Optimizer	Adam
Loss function	Cross-entropy
Gradient Clipping	1.0

TABLE 3.2: Hyperparameters used for training Mamba on Speech Commands

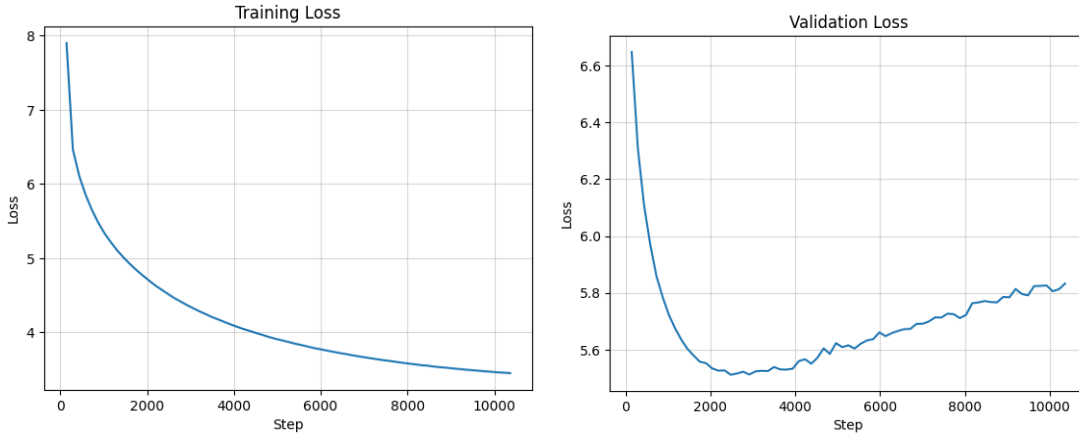


FIGURE 3.2: Mamba trained on Wikitext-2: training loss (left), validation loss (right)

3.1.1.3 WikiText-103

The WikiText-103 dataset contains over 100 million tokens from verified articles on Wikipedia. We have chosen WikiText-103 as our textual dataset because it contains long-range, coherent passages that preserve punctuation, casing, and paragraph structure, making it well-suited for assessing a model’s ability to capture contextual dependencies over extended sequences. Furthermore, as Mamba is being explored as a potential alternative to transformer-based large language models, we want to analyse how Mamba processes and represents linguistic information.

There is a smaller dataset called the Wikitext-2, which contains 2 million tokens from Wikipedia. The reason why we did not use the smaller dataset is that during training, the validation loss drops quickly initially, but it grows linearly as the training loss decreases, see [Figure 3.2](#). We have concluded that this shows the model overfitting the training dataset because of its small size, as such behaviour is not observed when the model of the same configuration is trained on Wikitext-103. Our goal is to train a model that has some understanding of the human language, not a model that only memorise the limited training examples. Therefore, WikiText-103 was selected as the main dataset, as it provides a larger and more diverse corpus to assess Mamba’s ability as a language model while remaining computationally manageable for our experiments.

Tokeniser We used the GPT-2 tokeniser from [Radford et al.](#). It is a Byte Pair Encoding (BPE) system that converts raw text into numerical tokens that can be used to train language models. It is a pre-trained tokeniser that merges frequently occurring character sequences into tokens, and it also distinguishes between leading spaces and character sequences. For example, the word "lowest" gets split into "low" and "est". These tokens are then converted into integer IDs and fed into the model. With a fixed

vocabulary size of 50,257 tokens, it is a reliable and widely adopted choice for training language models.

Training on WikiText-103 The data is first tokenised into integer tokens, and a single batch is shaped into a tensor of shape 32×256 with a batch size of 32 and block size of 256. The meaning of this shape is that, for each batch of tokens in the dataset, 256 tokens are concatenated into a 256-dimensional vector, and this is repeated 32 times. This is then fed into the model autoregressively, meaning that the data is input one token at a time, sequentially, so that the model cannot see the future and remain causally sound. The model outputs a tensor of the same shape, but with the next token predicted, the loss is then calculated with the target, which is also of shape 32×256 but contains the next token in all positions, which is just the original input shifted left by 1. This method of training aligns our selective state space model to generating texts by continuing sentences.

Hyperparameters:

Parameter	Value
Learning rate	3×10^{-4}
Batch size	32
Epochs	10
Weight decay	0.01
Model dimension	512
State dimension	128
Number of layers	8
Block size	512
Temperature	1.0
Optimizer	AdamW
Loss function	Cross-entropy
Tokeniser	GPT-2

TABLE 3.3: Hyperparameters used for training Mamba on WikiText-103

3.1.1.4 Selective Copying

The Selective Copying dataset by [Arjovsky et al. \(2015\)](#) is a synthetic benchmark designed to evaluate a model’s ability to perform selection and content-aware reasoning from long input sequences. In this task, the model is given a sequence of numbers containing mainly the "ignore" token, which is normally 0, referring [Figure 2.5](#). The

model is required to find all the non-zero numbers, the "memorise" tokens in the sequence and output them in the order they appeared in the input. For example, for the input "0010020304", the target would be "1234". Unlike real-world datasets, synthetic datasets are much less complex and less noisy, making them particularly suitable for interpretability, allowing us to directly observe when and how the model attends to important signals.

Training on Selective Copying A vector of length 4112 is fed into the model, containing 4096 zeroes as "ignore token", and 16 "memorise" tokens. Then the output of the model is a 1×16 vector of the 16 memorised tokens, and this is compared to the target with Cross-Entropy loss.

Hyperparameters:

Parameter	Value
Learning rate	10^{-4}
Batch size	64
Epochs	400000
Model dimension	64
State dimension	16
Number of layers	2
Number of noise	4096
Number of memorise	16
Optimizer	Adam
Loss function	Cross-entropy

TABLE 3.4: Hyperparameters used for training Mamba on Selective Copying

3.1.1.5 Induction Heads

The Induction Heads dataset is a synthetic dataset to assess a model's ability to recognise repeated patterns within a sequence, such as the word "Eiffel" often followed by the word "Tower". The reader can refer to [Figure 2.5](#) for a simplified diagram of the task. The task consists of an input sequence with some repeated patterns, and the model has to predict the correct token the next time it sees the same pattern. For example, for the input "123412001", the model has to predict the number "2" as the pattern "12" has repeated twice. This dataset is useful for analysing how Mamba remembers and recognises repeated patterns and uses them to predict what comes next.

Training on Induction Heads The input is a sequence of numbers containing certain repeated two-number patterns, with the last number being the first number of the pattern. The model needs to predict the correct number, i.e., perform next-token prediction on the last number.

Hyperparameters:

Parameter	Value
Learning rate	0.001
Batch size	8
Epochs	1000
Model dimension	64
State dimension	16
Number of layers	2
Sequence length	256
Optimizer	Adam
Loss function	Cross-entropy

3.1.2 Obtaining and storing data

CIFAR-10, Speech Commands, and Wikitext datasets are publicly available and are downloaded via widely available Python APIs, using PyTorch, TensorFlow, and datasets api respectively. The downloaded datasets will be stored on Iridis under a password-protected account. Synthetic datasets are generated at runtime and no data is stored.

3.2 Model training and evaluation

3.2.1 Iridis HPC cluster

For this project, training and experiments are conducted on the Iridis high-performance computing (HPC) cluster from the University of Southampton. It provides essential compute necessary for training and evaluating deep learning models at scale. Since we are training Mamba across multiple datasets, including image, audio, text, and synthetic sequence tasks, the use of Iridis enabled efficient parallelisation, faster training times, and remote storage of datasets and model checkpoints. By utilising Iridis, computations can be executed remotely without relying on local computing resources, allowing training scripts to run independently of local machines, even when my local device was turned

off. Access to GPU-enabled nodes were particularly important for me as I do not have access to GPU training locally.

Access to the Iridis cluster is established through SSH (Secure Shell), which enables the user to connect to the system from our local machines if they are on the university network or through a VPN (Virtual Private Network). Once connected, users interact with the cluster through a command-line interface in Linux. To train our models, datasets are first downloaded by running the script on the login nodes, since the compute nodes do not have access to the internet. Then, the task is submitted to a job scheduling system called SLURM using a shell script and the command `sbatch`. The job will run until it is completed or the time limit is reached.

3.2.2 Training results

Below are the training results of our best models on the validation dataset:

	Speech Commands		CIFAR-10		WikiText-103		Synthetic accuracy	
	Acc	Loss	Acc	Loss	PPL	Loss	Copy	Induction
Training	1.0	0.0352	0.6018	1.133	20.656	3.028	1.0	1.0
Validation	0.9701	0.0915	0.5385	1.322	22.58	3.117	0.1064	1.0

TABLE 3.5: Training results across tasks

3.3 Mamba implementation

We used an implementation by [Ran-Milo et al. \(2024\)](#), which is based on the original implementation from [Gu and Dao \(2023\)](#). The main advantages of this implementation are the implementation of complex parametrisation of the SSM block and the option to swap to S4D inside the Mamba block instead of S6, swapping the SSM from time-varying to time-invariant. The architecture is similar to [Figure 2.10](#).

3.3.1 Classification architecture

The classification architecture used in this project is based on a simple yet effective pipeline built around the Mamba backbone ([Figure 3.3](#)). Input data is first passed through an initial linear projection layer, which maps the raw input into the model’s dimension. This is followed by the Mamba model, which models the sequence, capturing

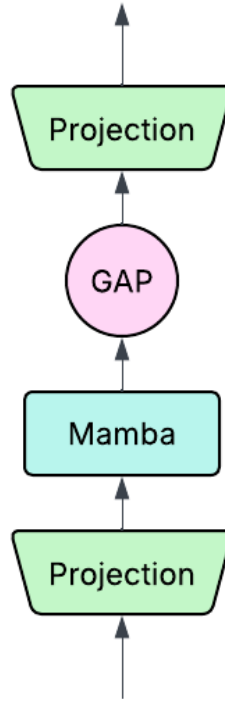


FIGURE 3.3: Mamba classification model

temporal dependencies. To aggregate the entire length of the sequence of representations produced by Mamba, a Global Average Pooling (GAP) layer is applied, which calculates the mean over the entire length of the representation for each dimension. Finally, a second linear projection layer maps the pooled vector to the desired output, producing class logits for classification. This pipeline provides a simplistic and interpretable pipeline, allowing us to isolate the SSM and analyse how information is transformed, while maintaining computational efficiency and performance.

3.3.2 Generation architecture

The architecture for generation tasks is also a simple pipeline with an embedding layer [Figure 3.4](#). The Mamba block performs sequence modelling and its output is projected and used to predict the next token at every time step.

At inference time, after the model has predicted the next token, the last token of the prediction is input back into the model while the hidden state is cached. Since Mamba is a state space model, the cached hidden state contains all the information of the past tokens so the past tokens are no longer needed. The last token is processed into a state representation and is used to predict the next token. This process is repeated until

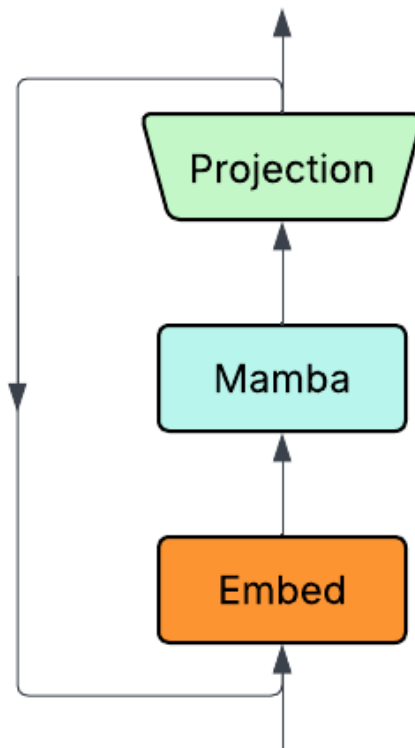


FIGURE 3.4: Mamba generation architecture

the max number of tokens is reached. This is how Mamba achieves linear time during inference.

Chapter 4

Methodology

4.1 Motivation for interpreting Mamba

Modern deep sequence models such as (transformers, RNNs, SSMs) are often treated as black boxes, it is often very difficult to grasp what these models have learned during training. Interpretability is crucial for examining the explainability, trustworthiness and robustness of the model. These factors are important for to use these models in critical industries such as health care and manufacturing.

Mamba is not just another machine learning architecture, it has transformer-level performance on the tasks that transformers excel at such as language, while having linear time inference. It achieves this by using state space rather than attention, compressing pass information into a state representation. It overcomes the slow training time of traditional RNNs by utilising a hardware-aware algorithm and incorporate time-varying state transition, improving from previous SSMs.

Since SSMs have constrained their state space to just a constant dimension, it is considered difficult to interpret them compared. In this chapter, we propose a set of analysis tools and explain our motivation to use them in studying the behaviour of LTV state space models, with the evaluation results in the next chapter ([chapter 5](#)).

4.2 Eigenvalue-Based Analysis

4.2.1 Analysis of Eigenvalue trajectories

Gu and Dao (2023) have shown that the selective SSM is connected to the RNN gating mechanism such that when $g_t \rightarrow 1$, $x_t \rightarrow u_t$, and when $g_t \rightarrow 0$, $x_t \rightarrow x_{t-1}$:

When the state dimension $N = 1$, $A = -1$, $B = 1$, $s_\Delta = \text{Linear}(u)$ and $\tau_\Delta = \text{softplus}$, then the SSM takes the form of

$$\begin{aligned} g_t &= \sigma(\text{Linear}(u_t)) \\ x_t &= (1 - g_t)x_{t-1} + g_t u_t \end{aligned} \tag{4.1}$$

They have stated that Δ can be viewed as a generalisation of Equation 4.1. Essentially, Δ controls how much to focus or ignore the current input u_t . For the continuous time SSMs from Equation 2.8 that are discretised by the time step Δ , when $\Delta \rightarrow \infty$ the model focuses on the current input more while $\Delta \rightarrow 0$ means that the model ignores the current input.

We propose that we can investigate the selectivity by analysing the eigenvalues of the $\bar{A}$ and $\bar{B}$ matrices as these determines the state dynamics as shown in Equation 4.1. These analysis will be carried out on more structured datasets such as selective copying and induction head as the resulting models trained on these datasets are simpler to interpret.

We have also animated the eigenvalues and made it into a mini-movie by recording a frame at every time step. This will be shown during the presentation of the project.

4.2.2 Stability Criteria

A digital filter h is a filter that processes a signal u in discrete time by calculating the convolution between the signal and the filter:

$$y[n] = \sum_{k=-\infty}^{\infty} h[k]u[n-k] \tag{4.2}$$

The behaviour of a digital filter is defined by its impulse response, which is the result of the convolution when u is:

$$\delta[n] = \begin{cases} 1 & \text{if } n = 0 \\ 0 & \text{if } n \neq 0 \end{cases} \tag{4.3}$$

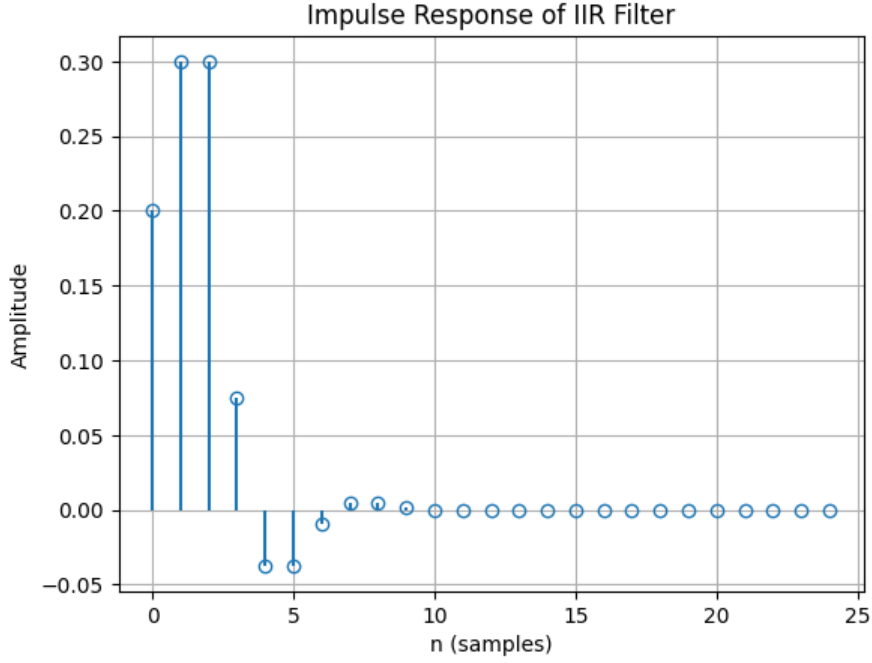


FIGURE 4.1: Impulse response of an IIR filter

A digital filter can be a finite impulse response (FIR) filter or an infinite impulse response (IIR) filter. FIR filters are filters with an impulse response within a finite time frame. On the other hand,, IIR filters produces an infinite impulse response. These IIR filters have the form of:

$$y[n] = \sum_{i=0}^P b_i x[n-i] + \sum_{i=1}^Q a_i y[n-i] \quad (4.4)$$

The feedback term $\sum_{i=1}^Q a_i y[n-i]$ makes the filters produce an infinite impulse response, see [Figure 4.1](#).

IIR filters can be represented by the discretised state space equation ([Equation 2.10](#)). That means that SSMs are IIR filters, with their impulse response being:

$$h[n] = \bar{C} \bar{A}^n \bar{B} \quad (4.5)$$

This is analogous to the convolution representation of the S4 kernel in [Equation 2.13](#). In SSMs, $\bar{A}$ is often a diagonalisable matrix $\bar{A}_n = \lambda_n$, and the diagonal elements of $\bar{A}$ are the eigenvalues of $\bar{A}$ and determines the growth of the system. Since [Equation 4.5](#) involves the powers of $\bar{A}$, this means the system will grow exponentially if $|\lambda_n| > 1$, see [Figure 4.2](#).

The constraints in Mamba constrained the magnitude of the eigenvalues of $\bar{A}$ to be less

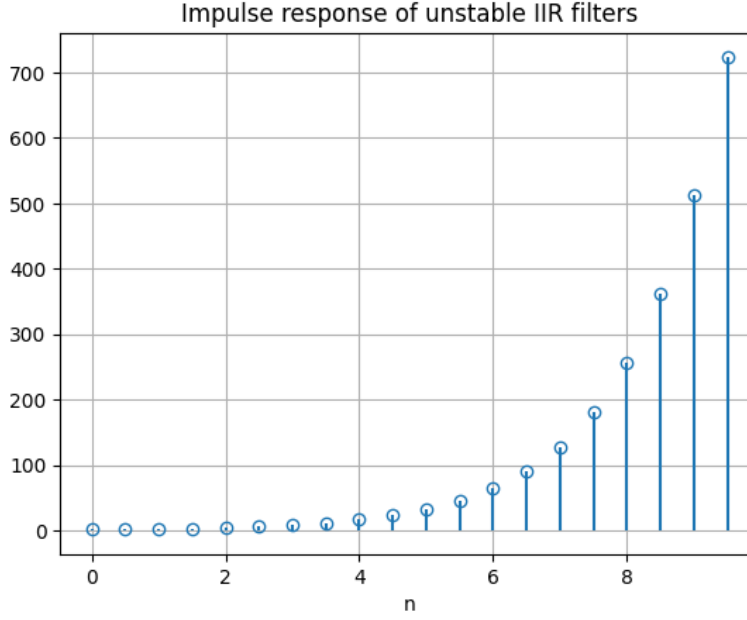


FIGURE 4.2: Impulse response of an unstable IIR filters

than 1 in [Equation 2.19](#) and [Equation 2.21](#). Eigenvalues lying within the unit circle in a complex plane guarantees the impulse response of the model to be exponentially decaying instead of growing. This ensures that the model's hidden states do not grow exponentially for arbitrarily long sequences, increasing the stability of the model.

In order to investigate the stability of Mamba, we have to extract the discretised matrix $\bar{A}$. Since $\bar{A}$ is a diagonal matrix, the values of the matrix are the eigenvalues. We have already proven analytically in [subsubsection 2.2.4.1](#) and [subsubsection 2.2.4.2](#) that Mamba constraints the magnitude of the eigenvalues of $\bar{A}$ to be less than 1. However, the real model implementation has not yet been empirically validated to confirm this stability property. We will investigate the stability of Mamba in [section 5.1](#).

4.3 Sensitivity and temporal dependencies

Understanding how sequence models utilise past information is crucial to interpretability. Transformer-based models ([Vaswani et al., 2017](#)) provide explicit attention weights that directly shows the dependencies between tokens, but the mechanisms of SSMs such as Mamba ([Gu and Dao, 2023](#)) is not as obvious. Instead, dependencies are implicitly encoded within their recurrent states.

In order to analyse these dependencies, we adopt a sensitivity-based framework, which measures how perturbations would influence the outputs of the SSM. This provides a meaningful and principled way to uncover the temporal structure of information flow

in Mamba. In this section, we will study the variance of eigenvalues and the Jacobian matrix of the SSM.

4.3.1 Eigenvalue variance as a temporal metric

Eigenvalues play a central role in SSMs; they determine the behaviour of the system. We can analyse how sensitive the model is to the input by determining how the system changes over time. In order to measure this, we can compute the eigenvalue variance at each time step:

$$\begin{aligned}\text{Var}(\lambda_t) &= \sum_{k=t}^{t+w} \frac{(\lambda_k - \mu)^2}{w} \\ \mu &= \frac{\sum_{k=t}^{t+w} \lambda_k}{w}\end{aligned}\tag{4.6}$$

This is the variance of the eigenvalue λ at time t over a sliding window of size w to reduce noise. We can produce the plots of eigenvalue variance over time by calculating the mean of all eigenvalues at each time step, the final results are shown in [Figure 5.5](#) and in [Appendix A.2](#).

4.3.2 Jacobian-based analysis

The Jacobian matrix is a fundamental concept in machine learning and multivariable calculus that describes all first-order partial derivatives of a set of variables with respect to another set of variables. For example, for the vector-valued function $y = f(x)$, where both x and y are vectors, the Jacobian is defined as the matrix of all partial derivatives:

$$J = \frac{\partial y}{\partial x}\tag{4.7}$$

Each element $J_{i,j}$ represents the partial derivative of the i -th y with respect to the j -th x , i.e., $J_{i,j} = \frac{\partial y_i}{\partial x_j}$. As such, the Jacobian measures the local sensitivity of the system, indicating how small perturbations to x affect the y .

In the context of state space models, the Jacobian provides us with a good way to analyse sensitivity and temporal dependencies. By measuring how the state of the SSM y_t , responds to changes in the adjustable time step $\Delta_{t'}$, i.e., $\frac{\partial y_t}{\partial \Delta_{t'}}$, we can understand how sensitive the models are to Δ and captures how temporal dependencies propagate through the model.

4.3.2.1 Efficient computation of Jacobians for Mamba

A naive approach to computing the Jacobian will be to compute each value one by one by the chain rule and form the final matrix by concatenating all the values. However, for long sequences such as the Speech Commands data, which contains 16000 time steps, the resulting matrix would be a 16000×16000 matrix, this will require 256 million computations to calculate the final matrix! To make the computation of the Jacobian possible, we need a numerically stable and vectorised approach.

To address this, we have derived a formulation that utilises vectorised and GPU acceleration by avoiding repeated computations. The following is the derivation of such method:

We start from the discretised state space equations, using the equation in [subsection 2.2.4.4](#) to discretise B .

$$\begin{aligned} x_t &= \bar{A}_t x_{t-1} + \bar{B}_t u_t \\ &= e^{\Delta_t A} x_{t-1} + \Delta_t B_t u_t \\ y_t &= C_t x_t + D u_t \\ &= C_t (e^{\Delta_t A} x_{t-1} + \Delta_t B_t u_t) + D u_t \end{aligned}$$

And we derive the formulation by the chain rule of derivatives.

$$\begin{aligned} \frac{\partial y_t}{\partial x_t} &= C_t \\ \frac{\partial x_t}{\partial x_{t'}} &= \prod_{k=t'}^t \frac{\partial x_{k+1}}{\partial x_k} \\ &= \prod_{k=t'+1}^t \bar{A}_k \\ \frac{\partial x_{t'}}{\partial \Delta_{t'}} &= \partial(e^{\Delta_{t'} A} x_{t'-1} + \Delta_{t'} B_{t'} u_{t'}) / \partial \Delta_{t'} \\ &= A \bar{A}_{t'} x_{t'-1} + B_{t'} u_{t'} \\ \frac{\partial y_t}{\partial \Delta_{t'}} &= \frac{\partial y_t}{\partial x_t} \cdot \frac{\partial x_t}{\partial x_{t'}} \cdot \frac{\partial x_{t'}}{\partial \Delta_{t'}} \\ &= C_t \cdot \left(\prod_{k=t'+1}^t \bar{A}_k \right) \cdot (A \bar{A}_{t'} x_{t'-1} + B_{t'} u_{t'}) \\ \frac{\partial y_t}{\partial \Delta_{t'}} &= \begin{cases} C_t \left(\prod_{k=t'+1}^t \bar{A}_k \right) (A \bar{A}_{t'} x_{t'-1} + B_{t'} u_{t'}) & t > t' \\ C_t (A \bar{A}_t x_{t-1} + B_t u_t) & t = t' \\ 0 & t < t' \end{cases} \end{aligned}$$

The part that is recomputed the most with the naive approach is the cumulative product of $\bar{A}$. With this formulation, all cumulative products can be precomputed by this with cp defined as a list of the cumulative products of $\bar{A}$ from 0:

$$cp = [\bar{A}_0, \bar{A}_0\bar{A}_1, \dots, \bar{A}_0 \dots \bar{A}_t] \quad cp^{-1} = [\frac{1}{\bar{A}_0}, \frac{1}{\bar{A}_0\bar{A}_1}, \dots, \frac{1}{\bar{A}_0 \dots \bar{A}_t}]$$

$$cp^T \cdot cp^{-1} = \begin{bmatrix} 1 & \frac{1}{\bar{A}_1} & \dots & \frac{1}{\bar{A}_1 \dots \bar{A}_t} \\ \bar{A}_1 & 1 & \dots & \frac{1}{\bar{A}_2 \dots \bar{A}_t} \\ \vdots & \vdots & \ddots & \vdots \\ \bar{A}_1 \dots \bar{A}_t & \bar{A}_2 \dots \bar{A}_t & \dots & 1 \end{bmatrix}$$

This can be vectorised in all N state dimensions with cp as a $T \times N$ matrix:

$$u^{T \times N} = C^{T \times N} \odot cp^{T \times N} \quad v^{T \times N} = (\frac{\partial x_{t'}}{\partial \Delta_{t'}})^{T \times N} \odot (cp^{-1})^{T \times N}$$

$$J_d^{T \times T} = u \cdot v^T$$

Where d indexes the model dimension, with one Jacobian per dimension. Below is a comparison of efficiency:

	Naive approach	Vectorised formulation
Asymototic complexity	$\mathcal{O}(T^2 N)$	$\mathcal{O}(T^2 N)$
Sequential depth	$\mathcal{O}(T)$	$\mathcal{O}(\log T)$
Time	3000 seconds	0.6 seconds

TABLE 4.1: Time efficiency comparison to calculate the Jacobian matrix for the Speech Commands dataset with $T = 16000$

4.4 Spectral analysis

Spectral analysis provides another perspective to time-domain analysis by examining how a model responds to different frequency components of an input signal. Understanding this behaviour is important for sequence modelling tasks, where inputs often contain high and low frequency patterns.

Traditional state space models, especially linear time-invariant systems, are well understood from a digital signal processing perspective. By applying tools such as the Fourier transform or the Z-transform, these systems can be represented through a transfer function. This interpretation has been widely used in digital signal processing to design control systems and digital filters, and analyse properties such as stability and filtering behaviour, e.g., high-pass or low-pass characteristics.

Deep state space models such as S4 (Gu et al., 2021) and (Gu and Dao, 2023) builds directly upon the foundation of classical state space models and extends them to deep learning with the introduction of structured state space. Mamba added more flexibility to the state space with input-dependency. However, this addition complicates the analysis, as the system is no longer strictly time-invariant. Despite this, studying the spectral properties of Mamba remains valuable, allowing us to approximate how the model processes frequency components in practice.

By linking Mamba to its roots in SSMs, spectral analysis provides a principled and well-studied framework for interpreting its behaviour. It enabled us to investigate the filtering properties of Mamba.

We will mainly focus on the dataset Speech Commands, as it has the deepest connection to signal processing, and it allows us to investigate how Mamba acts as a filter.

4.4.1 Z-transform Formulation

The Z-transform is a mathematical tool used to analyse discrete-time signals and systems. It is similar to the Laplace transform in continuous-time systems. Given a discrete signal x , its Z-transform is:

$$X(z) = \sum_{t=0}^{\infty} x_t z^{-t} \quad (4.8)$$

where t is the discrete time, z is a complex variable, and $X(z)$ is a function in the complex frequency domain.

This allows us to compute the transfer function:

$$H(z) = \frac{Y(z)}{U(z)} \quad (4.9)$$

where $Y(z)$ and $U(z)$ are the Z-transform of the output and the input signal respectively.

By adding the restriction that z lies on the unit circle, i.e., $z = e^{i\omega}$, the result will become the frequency response of the system. This is a standard result in linear systems theory (Lyons, 2011b).

4.4.2 Computation of Spectral Responses

To analyse the spectral behaviour of the model, we compute its Z-transform response as the model is a discrete-time model. For a classical discrete-time linear time-invariant

(LTI) state space model of the form:

$$x_t = \bar{A}x_{t-1} + \bar{B}u_t \quad y_t = Cx_t$$

The relationship between the input and the output in the Z-domain can be described by the transfer function similar to the one used in [Yu et al. \(2025\)](#):

$$H(z) = C(zI - \bar{A})^{-1}\bar{B} \quad (4.10)$$

Unlike classical LTI systems, Mamba introduces the $\bar{A}, \bar{B}, C$ matrices, which are input dependent. As a result, a single transfer function does not fully describe its function.

To address this, we approximate the spectral response by freezing the parameters in time. We consider the responses at specific time steps to capture local behaviour. Hence, the model is treated as locally time-invariant, allowing the following formulation to be used under the same transfer function formulation:

$$H_t(z) = C_t(zI - \bar{A}_t)^{-1}\bar{B}_t \quad (4.11)$$

where the matrices and the output are dependent on the input at time t .

4.4.3 Complex-valued models

Our analysis will mainly focus on the complex-valued version of Mamba, as it enables a more interpretable characterisation of the model's behaviour. With the complex parametrisation, the eigenvalues of the state transition matrix $\bar{A}$ can be directly related to the decay rate and the oscillation rate, with their magnitudes controlling the decay and their phases, determining the frequency.

In addition, classical spectral analysis tools such as the Z-transform and Fourier transform inherently operates in the complex domain. Furthermore, [Ran-Milo et al. \(2024\)](#) has proven that complex SSMs are strictly more efficient and learnable than real-valued ones, as complex parametrisation enables representation oscillating and decaying sinusoidal waves compared to sums of decaying exponentials in real-valued SSMs.

4.4.4 Frequency Response Analysis

The frequency response of a state space system provides a direct way to characterise how it processes different frequency components of an input signal. For S6/Mamba, the

frequency response is a special case of the Z-transform given by:

$$H_t(e^{i\omega}) = C_t(e^{i\omega}I - \bar{A}_t)^{-1}\bar{B}_t \quad (4.12)$$

where $\omega \in [0, 2\pi]$ and represents the angular frequency. Notice that the function $H(e^{i\omega})$ is periodic for every 2π as ω is the phase angle of the unit circle on the complex plane, the phase of the circumference of the unit circle circles back to 0 radians when it reaches 2π . The relationship between the angular frequency and the actual frequency for a discrete system is this:

$$f = \frac{\omega f_s}{2\pi} \quad (4.13)$$

where f_s is the signal sampling rate, for Speech Commands it is 16000 Hz. We visualise the magnitude $|H(e^{i\omega})|$ as it indicates how strongly the system amplifies or attenuates signals at different frequencies.

4.4.5 Z-domain Response Analysis

The Z-domain response/Z-transform provides a more complete characterisation of the model's spectral behaviour by extending the analysis to the entire complex plane. The Z-transform provides a wider range analysis and allows for stability analysis. It lets us visualise how the model behaves as a system, rather than only as a frequency filter.

In classical control theory, the Z-domain is used to analyse poles and zeros, which determine the system's behaviour. Poles are values of z where $H(z) \rightarrow \infty$. They are the points where the system response is the strongest. These define the system's stability, the poles must be inside the unit circles for it to be unconditionally stable. Zeros are values of z where $H(z) = 0$. Poles show the frequency response gain, and Zeros show the attenuation, the closer the pole or zero is to a unit circle, the stronger will be its effect at the frequency associated with its phase angle. These can refer to Chapter 6.5 of the book [Lyons \(2011b\)](#).

For the transfer function in [4.11](#), the poles occur when the denominator is 0, so:

$$\begin{aligned} zI - \bar{A} &= 0 \\ z &= \bar{A} \end{aligned} \quad (4.14)$$

This gives us another perspective to view the eigenvalues of $\bar{A}$ as the poles of the transfer function, which in turn determines the peaks in the frequency response.

Chapter 5

Evaluation

5.1 Results on model stability

The following are the plots of $\bar{A}$ eigenvalues for real and complex parametrised Mamba trained on the Speech Commands dataset. Plots of other datasets can be found in [Appendix A.1](#).

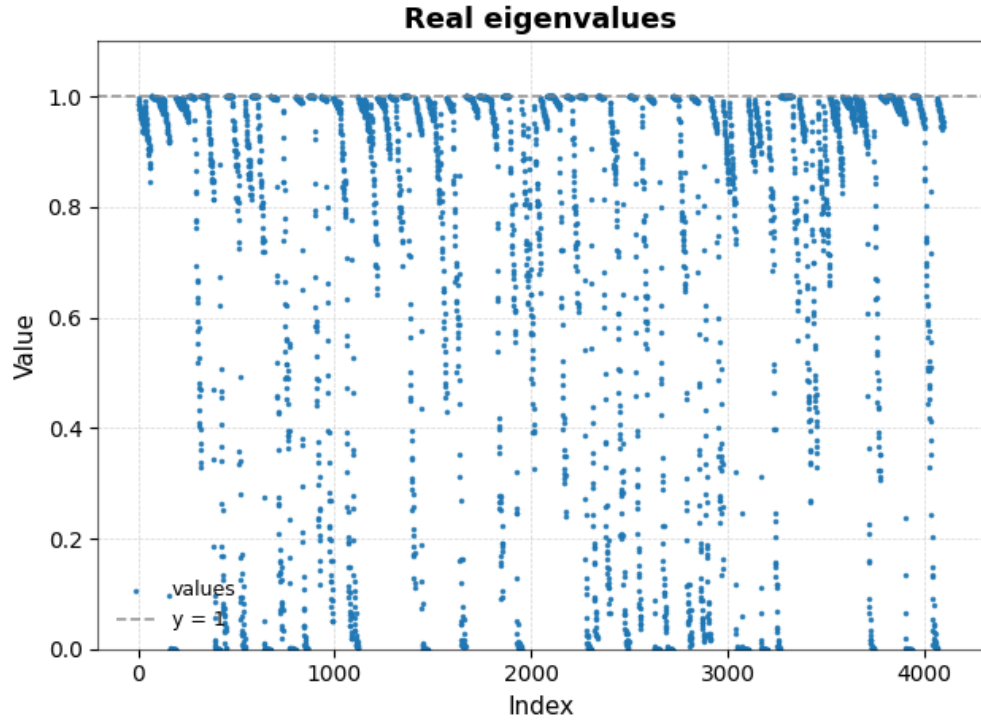


FIGURE 5.1: Eigenvalues of $\bar{A}$ for real-valued Mamba trained on the Speech Commands dataset

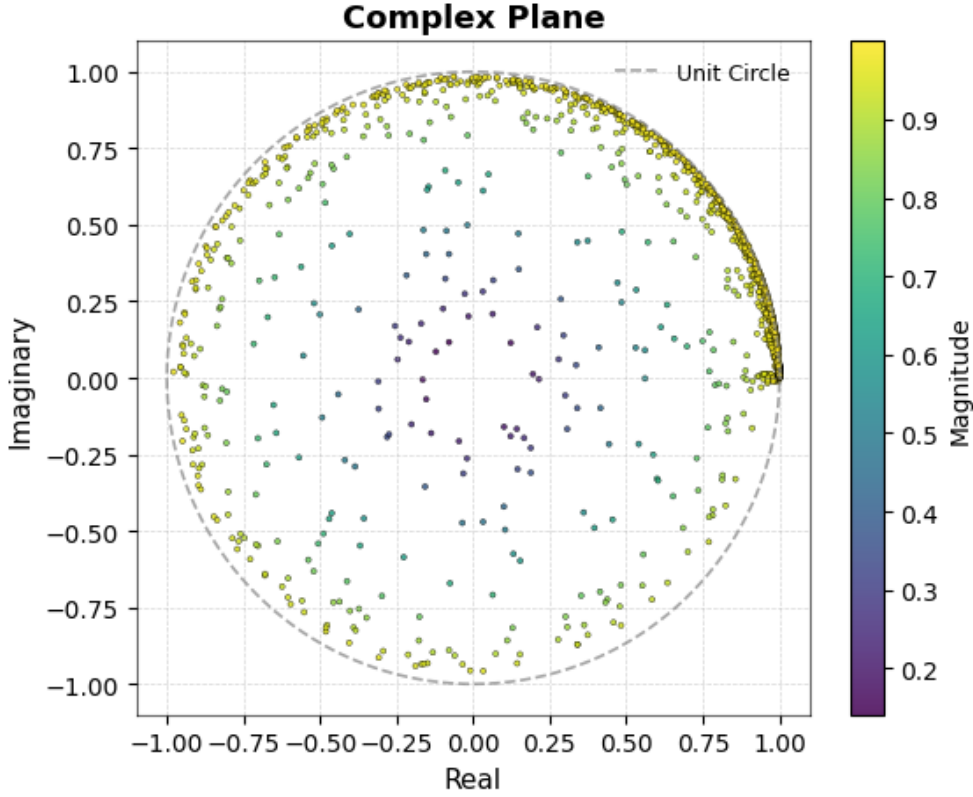


FIGURE 5.2: Eigenvalues of $\bar{A}$ for complex-valued Mamba trained on the Speech Commands dataset

Both plots show all eigenvalues of Mamba at one time step of one sample, more examples can be found in [Appendix A.1](#). The real-valued plots the values of the flattened $\bar{A}$ matrix with a $y = 1$ line at the top showing the constraint. The complex-valued plots the values on a complex plane with a unit circle showing the constraint of magnitude less than 1.

From the plots we can deduce the magnitude of the eigenvalues of $\bar{A}$ for real-valued Mamba are less than 1 and for complex-valued Mamba is within the unit circle. This empirical observation is consistent with the theoretical constraints derived in [subsubsection 2.2.4.1](#) and [subsubsection 2.2.4.2](#) are true and suggests that the stability guarantees imposed by the parametrisation are preserved in practice after training. In particular, even though the eigenvalues are learned through optimisation, the resulting values remain confined within the unit circle for both real and complex valued models, preventing explosive behaviour commonly associated with unstable IIR systems. We can conclude that Mamba is asymptotically stable over long sequences.

However, it is important to note that this analysis is performed on individual snapshots of trained models and specific datasets. A more comprehensive evaluation across a wider range of datasets would be necessary to fully assess the stability property of Mamba under all practical settings.

5.2 Results on eigenvalue analysis on synthetic datasets

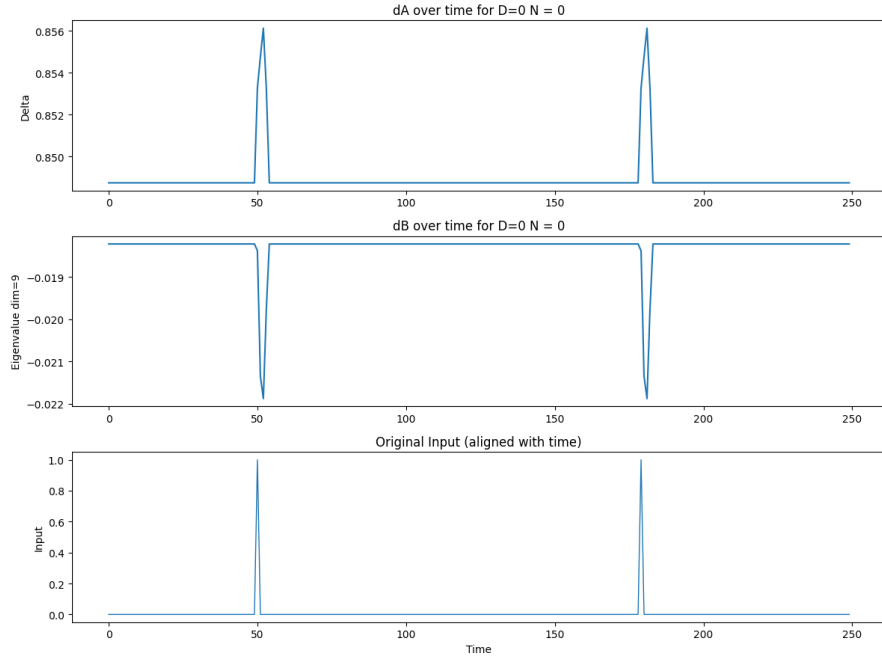


FIGURE 5.3: A slice of the eigenvalues after training on selective copying: $\bar{A}$ (top), $\bar{B}$ (middle) and the input (bottom)

In selective copying, we can clearly see that during period of ignore tokens, the model remains stable and constant, ignoring the noise. When the model sees a memorise token, $\bar{A}$ spikes upwards and remain high for a few time steps, while $\bar{B}$ spikes downwards during this period. This shows the gating mechanism of Mamba, when it sees an important token, $\bar{A}$ value is high remains high after the token to reduce the decay rate of the memory, $\bar{B}$ becomes more negative so that the input is written into the hidden state with more significant impact, acting like a remember and forget gate, as shown in [Equation 4.1](#).

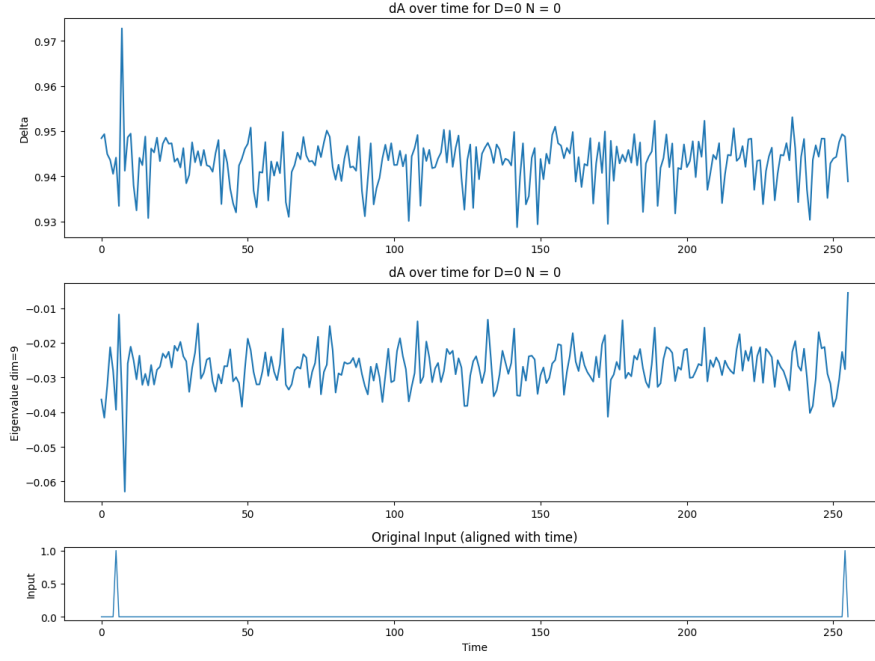


FIGURE 5.4: Eigenvalues after training on Induction Head: $\bar{A}$ (top), $\bar{B}$ (middle) and the input (bottom)

The same phenomenon can be observed in Mamba trained on the Induction Head dataset. When the model encounters the first token before the pattern, $\bar{A}$ spikes up, and at the next token (the token to memorise), $\bar{B}$ spikes down to write the token into memory.

We have analysed the eigenvalue trajectories of real-valued Mamba trained on synthetics datasets as this is the simplest form to analyse manually. Analysing eigenvalues on their own with more complex datasets such as Wikitext-103 is a much more difficult task and require studying multiple dimensions and layers. The resulting figures produced by the model shows that Mamba indeed undergo selective modelling, which is its main advantage over LTI SSMs.

5.3 Results on Eigenvalue variance

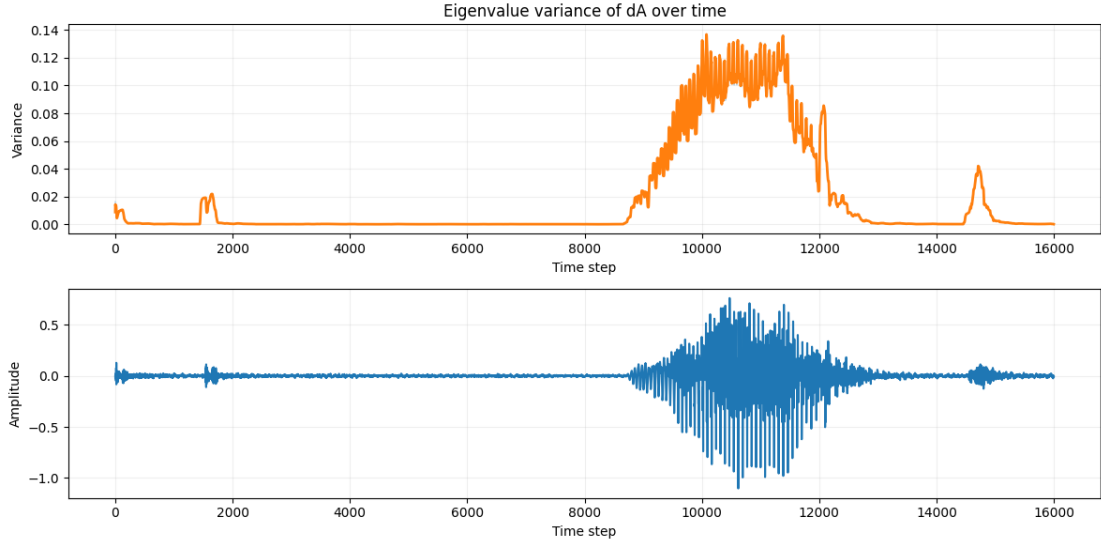


FIGURE 5.5: The plot of the eigenvalue variance of $\bar{A}$ on Mamba trained on Speech Commands (top). The original Speech Commands signal (bottom)

From Figure 5.5, we can observe that the variance is almost flat during silence periods, and high during utterances. During low variance periods, the system is operating in a consistent regime, where most modes behave similarly over time. This suggests that the model has learned to ignore noise and be stable in silence periods. When the speaker starts speaking, the model has learned to adapt and increase its variability to represent the complex speech patterns.

Compared to the figures of Wikitext in Figure 5.6, we observe that text-based sequences exhibit a more "spikey" variance over time, whereas speech signals tend to produce a smoother variance profile. We postulate that this is due to text signals being discrete, irregular, and much more information dense compare to the continuous signals of speeches. In conclusion, the variance plots reveal to us how much the SSM changes with respect to the input, giving us a quick and easy tool to assess their sensitivity.

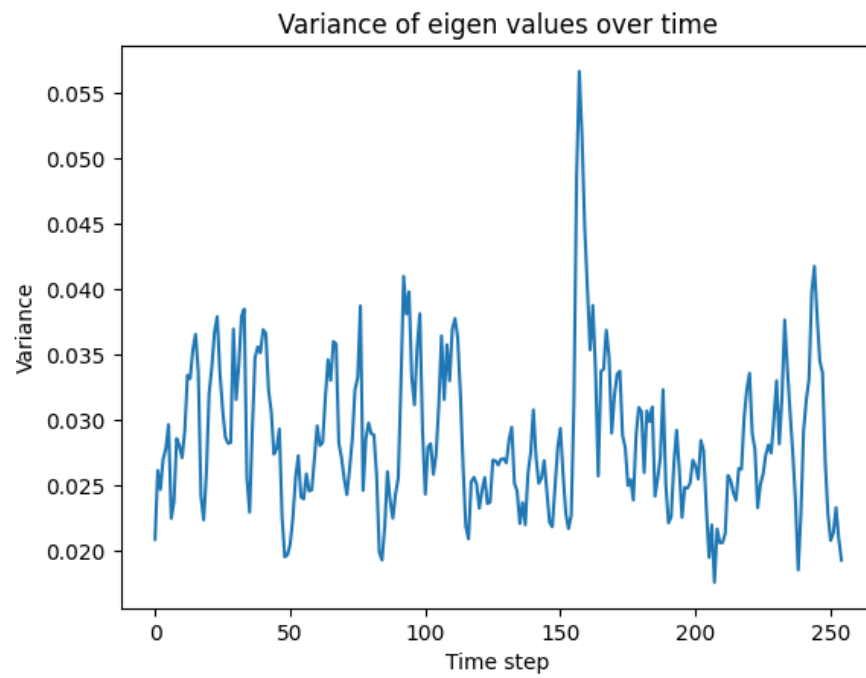


FIGURE 5.6: Eigenvalue variance for Wikitext-103

5.4 Results on Jacobian-based analysis

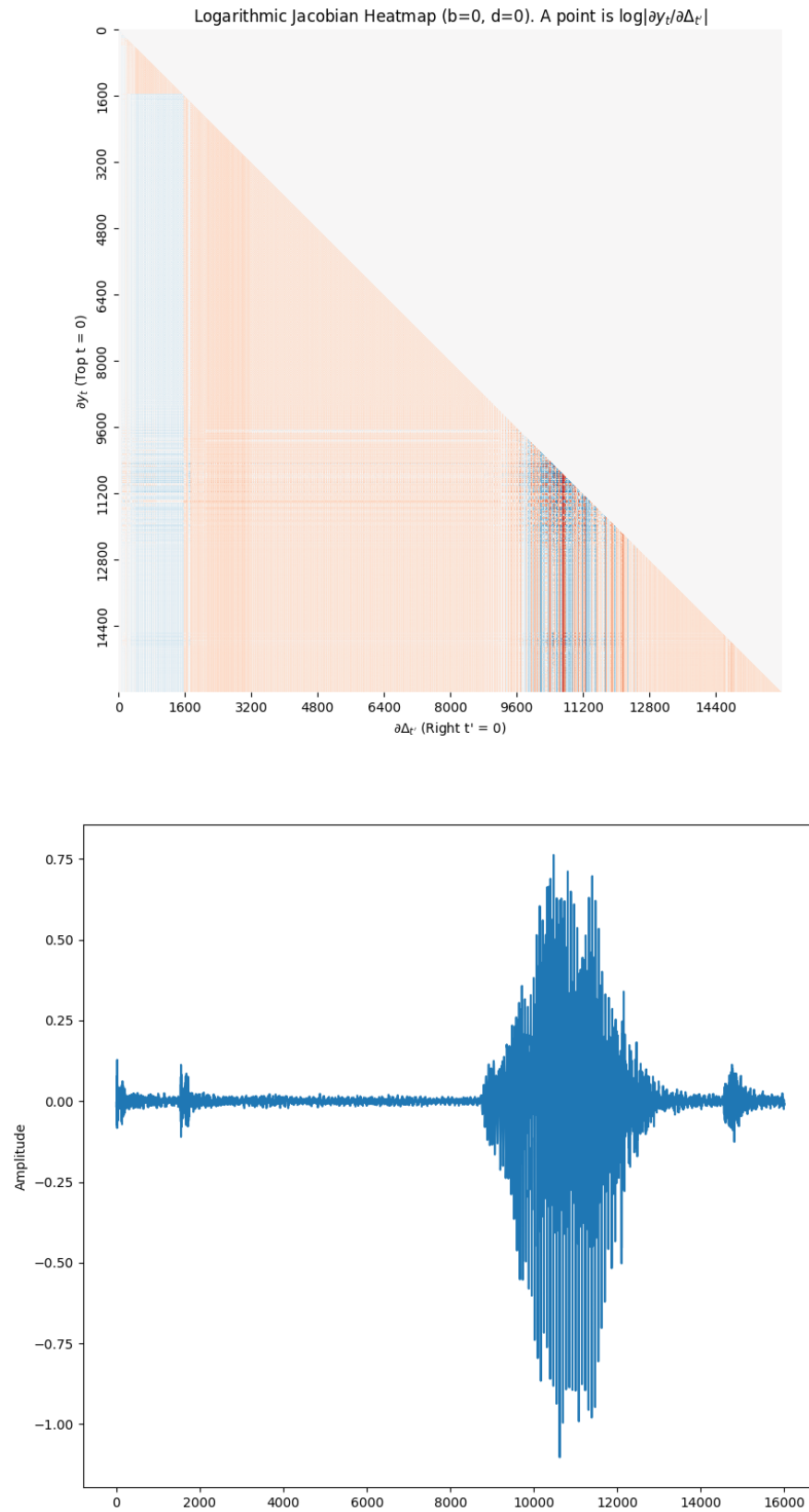


FIGURE 5.7: The Jacobian heatmap for Mamba trained on Speech Commands(top), the input speech signal (bottom), how this plot should be interpreted is described below

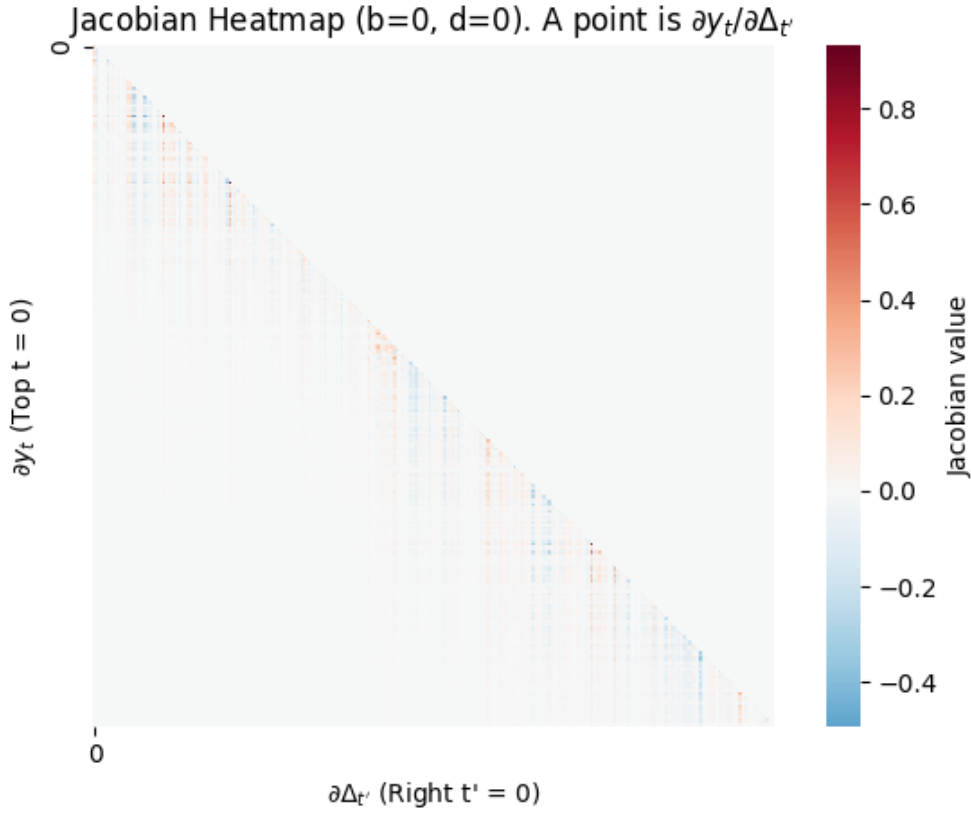


FIGURE 5.8: Jacobian plot for Mamba trained on Wikitext-103

The plot is read like this: for each point in the plot, its y-coordinate is t and x-coordinate is t' , the value of the point is $\frac{\partial y_t}{\partial \Delta_{t'}}$. The resulting Jacobian matrix is displayed as a heatmap with values of $(\log |\frac{\partial y_t}{\partial \Delta_{t'}}|)$ in this case to enhance contrast, the more red it is, the more positive, and vice versa for blue. The y-axis has $t = 0$ at the top, the x-axis has $t' = 0$ at the left.

The sensitivity of the model is given by the value of the partial derivative. For example, a column of the heatmap at t' is the Jacobian value of all the y for this specific $\Delta_{t'}$, a high value point at t implies a small change in $\Delta_{t'}$ has a big effect on y_t , meaning that the output is very sensitive to the time step. Since $\Delta_{t'} = \text{Linear}_1(u_{t'})$, it depends solely on the input signal u at time step t' , we can say that the sensitivity to $\Delta_{t'}$ is proportional to the sensitivity to $u_{t'}$.

Jacobian represents how a perturbation at time t' affects the output at time t . This allows it to serve as a dependency map over time, revealing how information propagates through the model. This enables the identification of short-range, long-range, and decaying dependencies.

By analysing a Jacobian plot, we can see how powerful it is. Figure 5.8 is the Jacobian plot for the first data for the batch ($b = 0$) and the first model dimension ($d = 0$), with

Mamba as a text generative model.

First of all, we can see that the upper-half triangle is empty, because the model is causal, so the SSM output y_t won't be able to see the future tokens at t' when $t' > t$. We can also see a faint diagonal line in the middle, which is the line where $t = t'$, since the state decays exponentially, we can expect the model to be most sensitive to the input the moment they first see it. This can be followed by looking at the columns of the heatmap, the colour goes fainter as it goes down, showing that as time goes on, the SSM output depends less on the specific input at t' , showing the memory decay property of SSMs.

Since Wikitext-103 is a text dataset, the input contains tokens that have different importance to the semantics of the text. The brighter columns can be interpreted as the model thinking the tokens at those columns are more important, hence the higher dependency on those time steps for the SSM outputs at future time steps. This is more obvious in [Figure 5.7](#), the aligned signal and Jacobian heatmap show that the colour is the brightest during the period of utterance, showing a strong dependency on the input signal at that time.

Overall, the Jacobian provides a powerful and theoretically grounded tool for analysing both sensitivity and temporal dependency within SSMs. We have shown a way to compute the Jacobian efficiently, and they have provided practical insights into how the model can be interpreted.

5.5 Results on Frequency Response

The resulting frequency response for the 10th model dimension at $t = 0$ for the same audio sample from [Figure 5.5](#) is available below and in [Appendix ??](#):

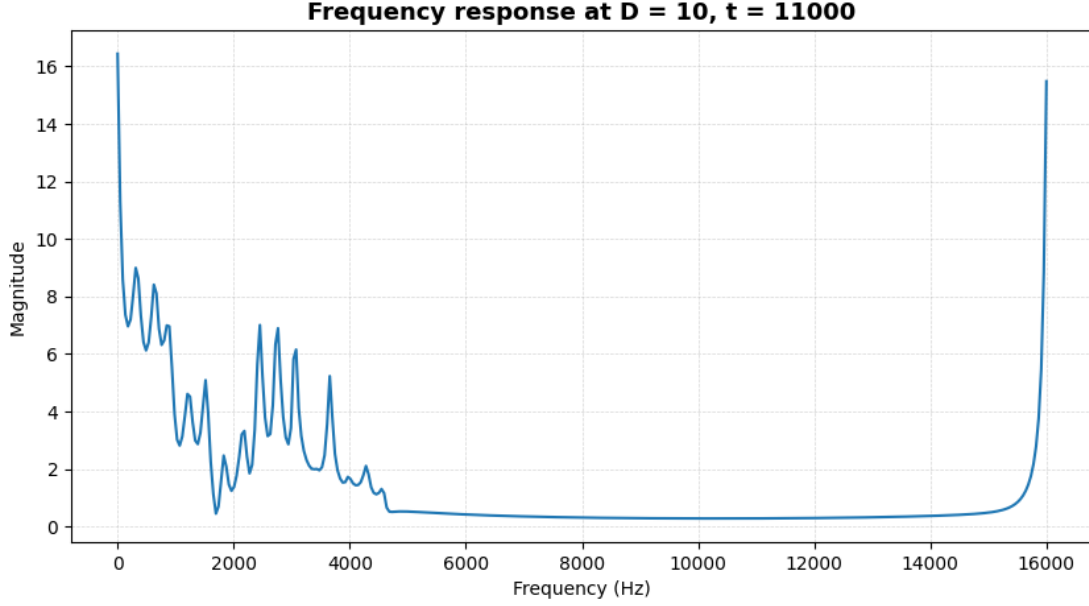


FIGURE 5.9: Frequency response for the dataset Speech Commands

Looking at [Figure 5.9](#), we can first see a strong peak at low frequency (0 – 500) Hz, this shows that the model is strongly amplifying slow and long-term components, this corresponds to the model prioritising long-term dependencies and slowly varying signals.

In the range of (500 – 4000) Hz, there are many irregular peaks, which have significant meanings. In English, the consonants (Q, R, S, T, etc.) are predominantly found in the frequency range matching the (500 – 4000) Hz range, matching the human speech intelligibility. The presence of multiple arrow-band peaks suggests that the model behaves as a set of filters, selectively responding to the signal.

After that, in the range (5–15) kHz, the frequency response is very low and flat, showing that the model suppresses high-frequency noise, focusing on more structured and stable signal components. The final peak around 16 kHz may be attributed to numerical effects caused by eigenvalues close to the unit circle near phase angle 0.

5.6 Results on Z-domain response

We have plotted $H(z)$ as a heatmap over the complex plane within a bounded region such as $[-1.2, 1.2] \times [-1.2, 1.2]$. For the same sample as [Figure 5.9](#), we obtained the following Z-transform heatmap:

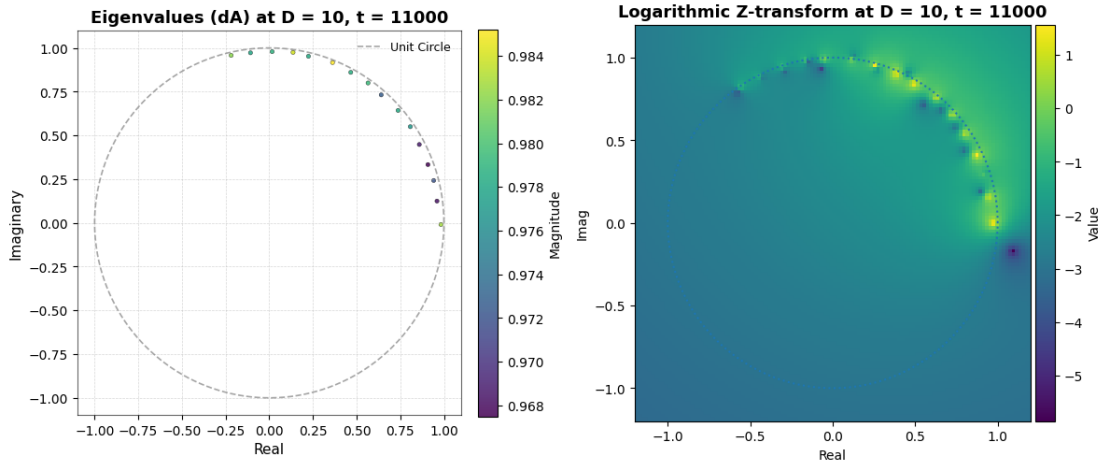


FIGURE 5.10: The eigenvalues of $\bar{A}$ (left). The logarithmic Z-transform for Speech Commands, $\log H(z)$, to enhance contrast (right)

In the Z-transform heatmap, the yellow dots are interpreted as poles, and the deep blue regions are interpreted as zeros. We can see that the location of the poles aligns with the eigenvalues on the left of Figure 5.10. For Mamba, we have found that after the initialisation scheme of poles near the unit circle, the model often maintains this property, with a preference for the upper right quadrant, as shown in more plots in Appendix ???. This indicates that the SSM prefers to persistent and low-frequency components for long-term memory.

The region within and outside of the unit circle is largely uniform and low in magnitude, indicating the model suppresses fast-decaying and unstable modes. The zeros in the Z-domain show the model is actively filtering out the frequencies associated with the phase angle of the zeros.

Overall, the Z-transform and the frequency response offer us a tool for interpreting Mamba as a state space system. By analysing the magnitude of $H_t(z)$, we can gain insight into the role of eigenvalues as poles, the mechanism of how S6 acts as an IIR filter, and its stability and amplification/attenuation behaviour. This connects the highly complex nature of deep state space model architecture to more understood classical state space theory, supporting the view that the SSMs inherit interpretable properties from traditional signal processing frameworks.

Chapter 6

Conclusions

6.1 Summary

This report set out to interpret the Mamba architecture, with the goal of bridging the interpretability gap of modern deep sequence models. By analysing Mamba beyond its performance, this work contributes toward a more principled understanding of how selective state space models process sequential data.

Our main contribution is in analysing the system matrices of the SSMs. We examined the time-varying behaviour and attempted to interpret the matrices on synthetic datasets. In addition, we investigated the sensitivity and temporal dependencies of the model through the lens of Jacobians. At last, a spectral perspective was adopted to interpret the model in the frequency domain.

The results demonstrate that Mamba exhibits highly adaptive, input-dependent behaviour, rather than behaving as a fixed linear system. In each section, we highlighted what each interpretation method reveals, showing that different analytical perspectives complement each other. The eigenvalue analysis provides insight into stability and selectivity, the Jacobian-based approach characterises sensitivity and temporal dependencies, while the spectral analysis offers an interpretation in terms of frequency-selective filtering.

Taken together, these perspectives present a coherent picture of Mamba as a sequence model, especially as a model for discrete data such as language and DNA. In conclusion, our research improves our understanding of LTV SSMs, but also demonstrates the value of combining classical signal processing theory with modern deep learning approaches to achieve more interpretable models.

6.2 Future works

Although this work has made attempts to interpret the internal mechanics of Mamba, several important directions remain for future research. The current study has mainly analysed one layer of Mamba, while it is common for Large Language Models (LLMs) have more than a hundred layers. Extending the analysis across multiple layers could reveal how the dynamics evolve with depth and whether hierarchical representations emerge.

Second, a promising direction lies in connecting Mamba more explicitly to classical Linear Time-Varying (LTV) system theory. This includes analysing its state transition matrix $\Phi(t, t_0)$, see [Equation 2.5](#), and computing its controllability and observability. Such tools could provide a stronger theoretical grounding for the model's behaviour since the current spectral analysis only focuses on local, one-step behaviour.

Another extension could be adding restrictions in the SSM, such as restricting eigenvalues, removing input dependency on certain matrices, and ablation studies. Combining these with the interpretability techniques we have developed would be an interesting direction and may reveal what each component in the SSM actually do.

Additionally, the Jacobian-based analysis could be extended to other sequence models, such as the RNN and Transformer, and compared to SSMs. In the frequency domain, time-varying frequency analysis could further characterise how Mamba alters itself to process signals. Finally, evaluating these methods on a broader set of data, such as the stock market and DNA sequences, may lead to more interpretable and robust SSMs.

6.3 Reflection

This project has been both technically and intellectually challenging and rewarding. Throughout the process, I developed a deeper understanding of both deep learning architectures and classical signal processing theory.

One of the main challenges encountered was training and fine-tuning the model on 5 datasets that are very different from each other. For example, when Mamba was training on Speech Commands, the model's performance was the same as random guessing due to exploding gradients, this is later solved by gradient clipping. From this experience, I learnt how to read results from other paper and stick to their configuration.

Furthermore, the help from my supervisor was invaluable throughout the project. Their feedback helped refine the technical direction and clarity of the analysis. Weekly meetings helped me overcome obstacles more efficiently and encouraged me to think more critically about my methodology and results.

Overall, the project has progressed smoothly throughout the year, It has significantly strengthened my ability to conduct independent research, and given me invaluable knowledge and experience in the field of machine learning.

Appendix A

Interpretation plots

A.1 Eigenvalue stability plots

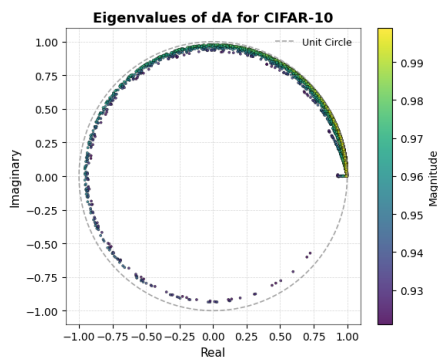


FIGURE A.1: Eigenvalues of $\bar{A}$ on Mamba trained on CIFAR-10

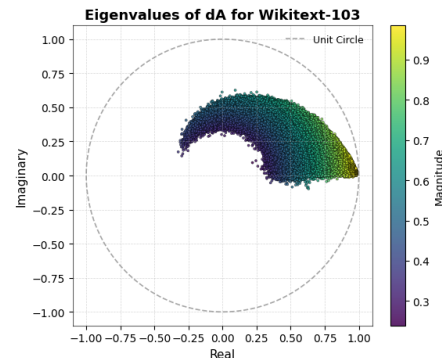


FIGURE A.2: Eigenvalues of $\bar{A}$ on Mamba trained on Wikitext-103

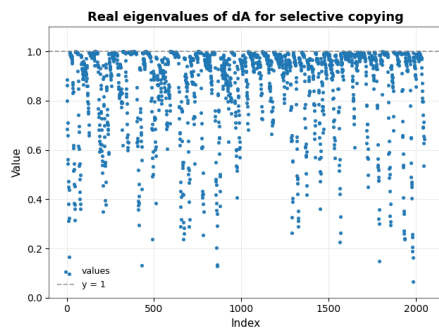


FIGURE A.3: Eigenvalues of $\bar{A}$ on Mamba trained on Selective Copying

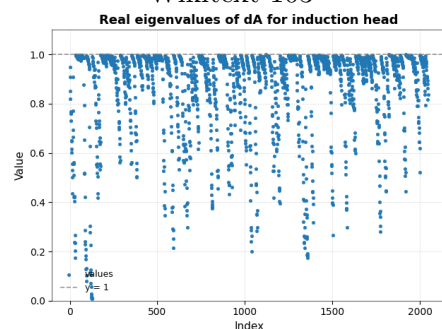


FIGURE A.4: Eigenvalues of $\bar{A}$ on Mamba trained on Induction Head

FIGURE A.5: Eigenvalues plot of $\bar{A}$ for complex (top row) and real (bottom row) Mamba trained on other datasets

A.2 Eigenvalue variance plots

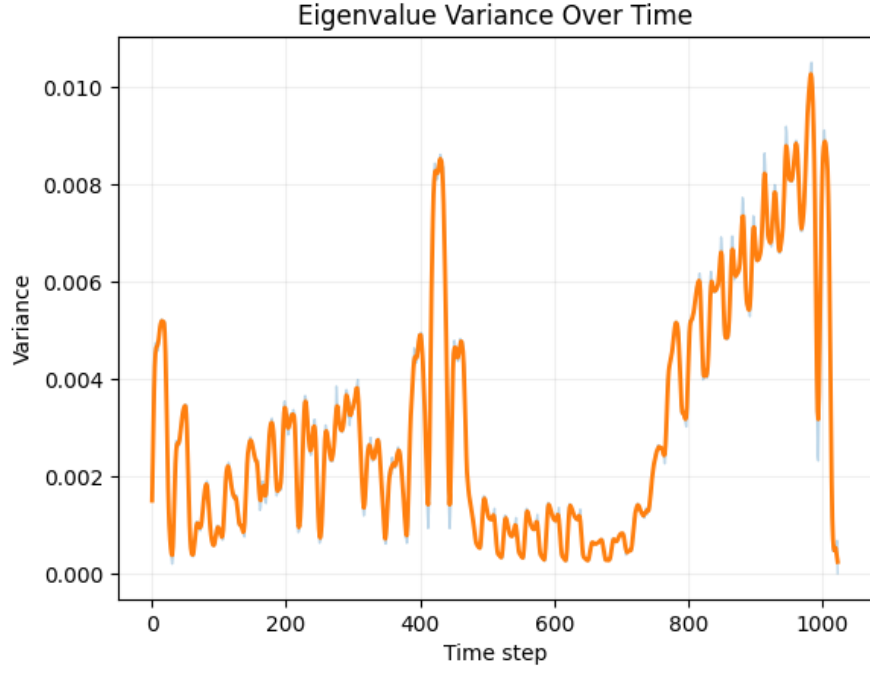


FIGURE A.6: Eigenvalue variance of $\bar{A}$ on Mamba trained on CIFAR-10

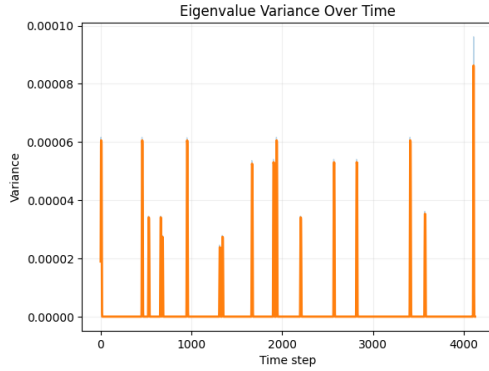


FIGURE A.7: Eigenvalue variance of $\bar{A}$ on Mamba trained on selective copying

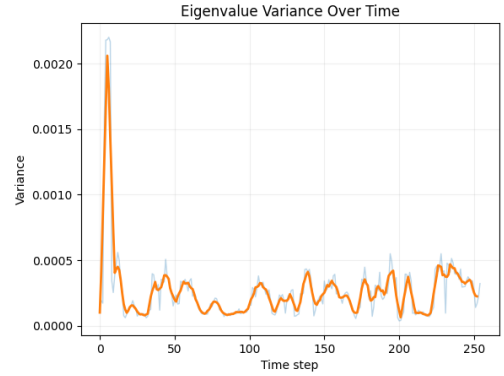


FIGURE A.8: Eigenvalue variance of $\bar{A}$ on Mamba trained on Induction Head

FIGURE A.9: Eigenvalue variance plot of $\bar{A}$ for other datasets

Appendix B

Project Management

B.1 Project Brief

Effects of filters in Linear Time Variant State Space Models

Name: Lok Hei Lam

Supervisor: Christine Evers

Problem Statement

Although linear time invariant SSMs (S4) are well-understood after thorough analysis of the filters, namely the A, B, C and D matrices, the behaviour of the model is less well-understood in linear time variant SSMs like Mamba.

Goal

Our goal is to build a conceptual bridge from the well-understood static filters of LTI models to the input-dependent parameters of LTV models, ultimately explaining how/why LTV models process information differently.

Scope

- **Model Analysis:** The models we will focus on are S4 for LTI models and Mamba for LTV models
- **Parameter Focus:** This investigation will centre on the classical A, B, C, D matrices from control theory, especially on B, C and Δ that became input-dependent in Mamba

- **Behavioural investigation:** Analyse and visualise the filters generated by LTV parameters for different inputs, probe the hidden states to understand how LTV matrices apply transformations analogous to filters in signal processing
- **Experiments:** Apply the analysis to small scale real-world data, such as image/audio classification or simple language modelling to observe the effects in a practical setting
- **Dataset:** Popular benchmark datasets for SSMs like CIFAR, LJSpeech, pathfinder, etc
- **Out of Scope:** This project is not focused on proposing new architectures or developing new algorithms or hardware for higher efficiency

B.2 Gantt Chart

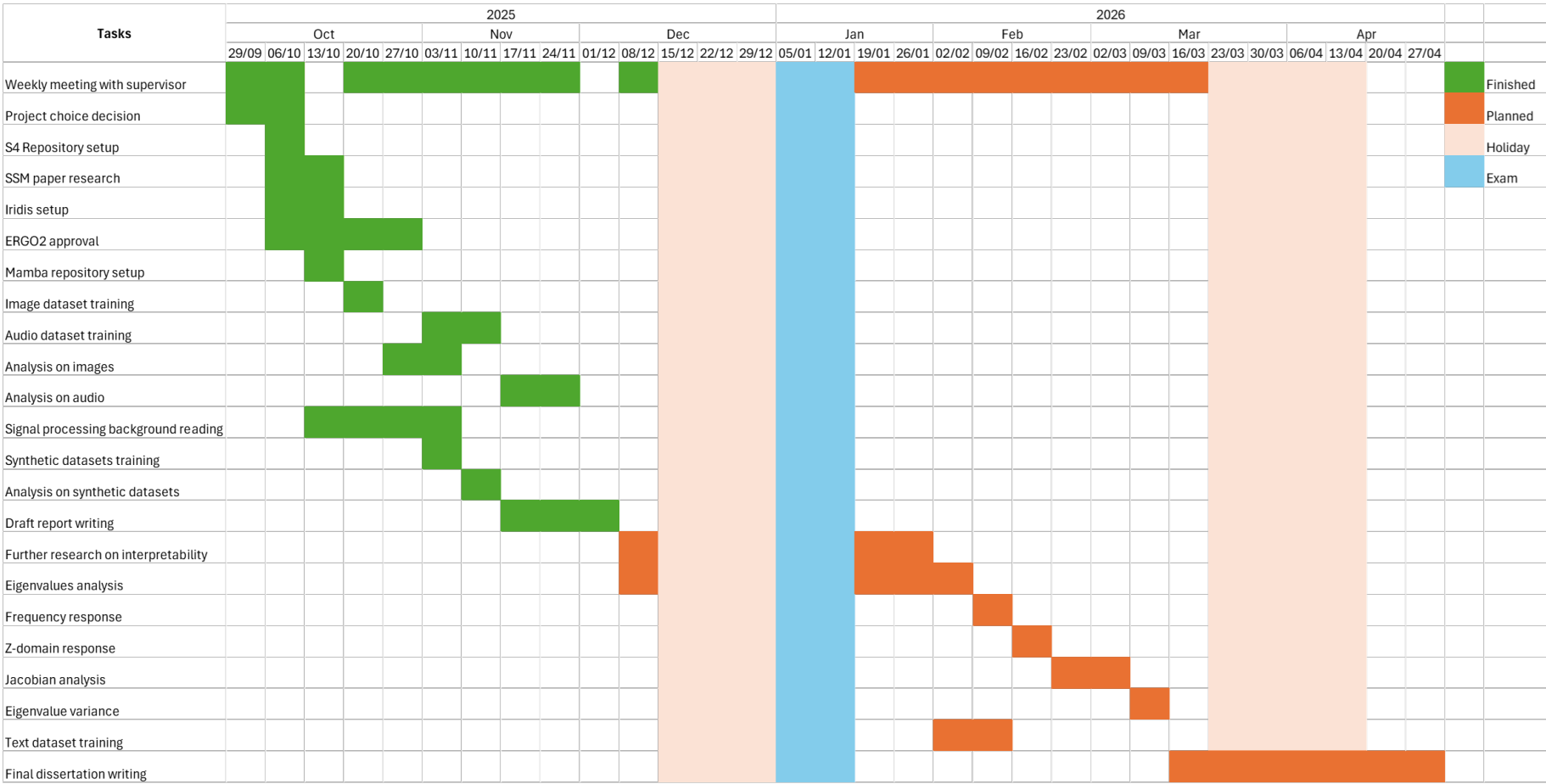


FIGURE B.1: Gantt chart for Semester 1

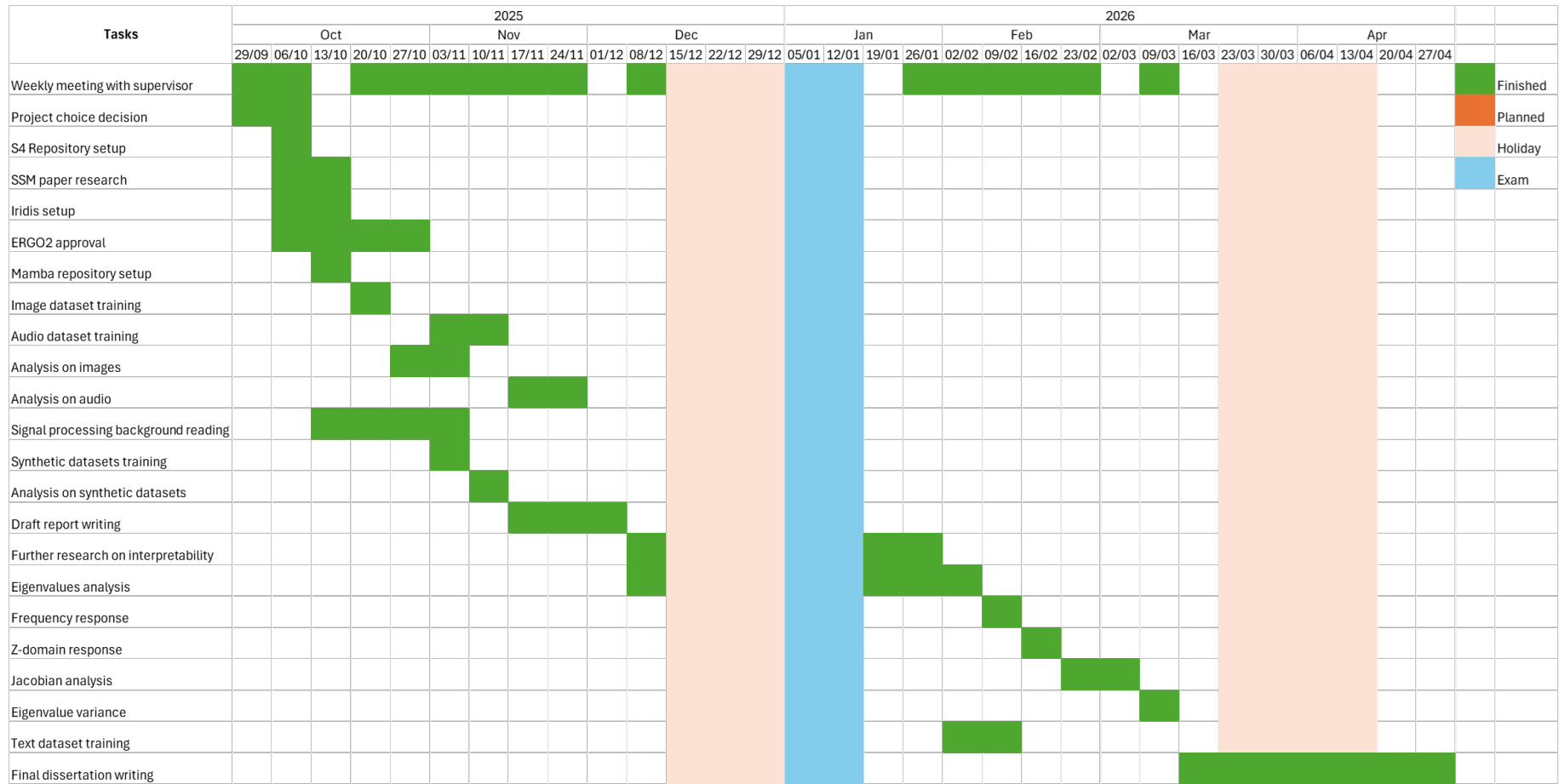


FIGURE B.2: Gantt chart for Semester 2

Bibliography

Martin Arjovsky, Amar Shah, and Yoshua Bengio. **Unitary evolution recurrent neural networks**. 11 2015.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. **Layer normalization**. 7 2016.

Dzmitry Bahdanau, Kyung Hyun Cho, and Yoshua Bengio. **Neural machine translation by jointly learning to align and translate**. *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings*, 9 2014.

Guy E Blelloch. Prefix sums and their applications.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. **Language models are few-shot learners**. *Advances in Neural Information Processing Systems*, 2020-December, 5 2020. ISSN 10495258.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. **Generating long sequences with sparse transformers**, 2019.

Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. **Rethinking attention with performers**, 2022.

Mohammed Dahleh, Munther A Dahleh, and George Verghese. Lectures on dynamic systems and control.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. **Flashattention: Fast and memory-efficient exact attention with io-awareness**. *Advances in Neural Information Processing Systems*, 35, 5 2022. ISSN 10495258.

- Tri Dao and Albert Gu. **Transformers are ssms: Generalized models and efficient algorithms through structured state space duality**. *Proceedings of Machine Learning Research*, 235:10041–10071, 5 2024. ISSN 26403498.
- dida. **What is an lstm neural network?**, 2025.
- Stefan Elfving, Eiji Uchibe, and Kenji Doya. **Sigmoid-weighted linear units for neural network function approximation in reinforcement learning**. *Neural Networks*, 107: 3–11, 11 2018. ISSN 18792782.
- David Eppstein, 2011.
- Daniel Y. Fu, Tri Dao, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. **Hungry hungry hippos: Towards language modeling with state space models**. *11th International Conference on Learning Representations, ICLR 2023*, 12 2022.
- Albert Gu and Tri Dao. **Mamba: Linear-time sequence modeling with selective state spaces**. 12 2023.
- Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. **Hippo: Recurrent memory with optimal polynomial projections**. *Advances in Neural Information Processing Systems*, 2020-December, 8 2020. ISSN 10495258.
- Albert Gu, Karan Goel, and Christopher Ré. **Efficiently modeling long sequences with structured state spaces**. *ICLR 2022 - 10th International Conference on Learning Representations*, 10 2021.
- Albert Gu, Ankit Gupta, Karan Goel, and Christopher Ré. **On the parameterization and initialization of diagonal state space models**. *Advances in Neural Information Processing Systems*, 35, 6 2022. ISSN 10495258.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. **Deep residual learning for image recognition**. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2016-December:770–778, 12 2015a. ISSN 10636919.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015b.
- Sepp Hochreiter and Jürgen Schmidhuber. **Long short-term memory**. *Neural Computation*, 9:1735–1780, 11 1997. ISSN 08997667.
- Zheyuan Hu, Nazanin Ahmadi Daryakenari, Qianli Shen, Kenji Kawaguchi, and George Em Karniadakis. **State-space models are accurate and efficient neural operators for dynamical systems**. 9 2024.

- Ningyuan Huang, Miguel Sarabia, Abhinav Moudgil, Pau Rodriguez, Luca Zappella, and Federico Danieli. [Understanding input selectivity in mamba: Impact on approximation power, memorization, and associative recall capacity](#). 6 2025.
- Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefer. [Data movement is all you need: A case study on optimizing transformers](#). 6 2020.
- Debasish Kalita. [What is recurrent neural networks \(rnn\)?](#), 2025.
- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. [Transformers are rnns: Fast autoregressive transformers with linear attention](#), 2020.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Scott Lundberg and Su-In Lee. [A unified approach to interpreting model predictions](#), 2017.
- Richard G. Lyons. *Understanding Digital Signal Processing*, chapter 5. Prentice Hall, Upper Saddle River, NJ, third edition, 2011a. ISBN 978-0137027415.
- Richard G. Lyons. *Understanding Digital Signal Processing*, chapter 6. Prentice Hall, Upper Saddle River, NJ, third edition, 2011b. ISBN 978-0137027415.
- Jun Ma, Feifei Li, and Bo Wang. [U-mamba: Enhancing long-range dependency for biomedical image segmentation](#). 1 2024.
- Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. [In-context learning and induction heads](#). 9 2022.
- Alan V. Oppenheim, Alan S. Willsky, and S. Hamid Nawab. *Signals & systems (2nd ed.)*, chapter 2. Prentice-Hall, Inc., USA, 1996. ISBN 0138147574.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. [Language models are unsupervised multitask learners](#).
- Yuval Ran-Milo, Eden Lumbroso, Edo Cohen-Karlik, Raja Giryes, Amir Globerson, and Nadav Cohen. [Provable benefits of complex parameterizations for structured state space models](#). *Advances in Neural Information Processing Systems*, 37, 10 2024.
- Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. ["why should i trust you?": Explaining the predictions of any classifier](#), 2016.

- Robin M. Schmidt. **Recurrent neural networks (rnns): A gentle introduction and overview**. 11 2019.
- Jimmy T.H. Smith, Andrew Warrington, and Scott W. Linderman. **Simplified state space layers for sequence modeling**. *11th International Conference on Learning Representations, ICLR 2023*, 8 2022.
- Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. **Long range arena: A benchmark for efficient transformers**. *ICLR 2021 - 9th International Conference on Learning Representations*, 11 2020.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. **Llama: Open and efficient foundation language models**. 2 2023.
- R. Tóth, F. Felici, P.S.C. Heuberger, and P.M.J. Van den Hof. **Crucial aspects of zero-order hold lqv state-space system discretization**. *IFAC Proceedings Volumes*, 41: 4952–4957, 1 2008. ISSN 1474-6670.
- Ashish Vaswani, Google Brain, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. **Attention is all you need**. page 1, 6 2017.
- Pete Warden. **Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition**. *arXiv preprint arXiv:1804.03209*, 2018.
- Annan Yu, Dongwei Lyu, Soon Hoe Lim, Michael W. Mahoney, and N. Benjamin Erichson. **Tuning frequency bias of state space models**. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. **Vision mamba: Efficient visual representation learning with bidirectional state space model**. *Proceedings of Machine Learning Research*, 235:62429–62442, 1 2024. ISSN 26403498.